



UNIVERSIDAD DE CHILE
FACULTAD DE CIENCIAS FÍSICAS Y MATEMÁTICAS
DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN

EXTENSIÓN EXPERIMENTAL DE APPLICATIVO EN GHC MEDIANTE
REORDENAMIENTO DE MÓNADAS CONMUTATIVAS EN PROGRAMAS
PROBABILÍSTICOS DE HASKELL

MEMORIA PARA OPTAR AL TÍTULO DE
INGENIERO CIVIL EN COMPUTACIÓN

ESTEBAN ROMERO BERRÍOS

PROFESOR GUÍA:
FEDERICO OLMEDO B.

PROFESOR CO-GUÍA:
ISMAEL FIGUEROA P.

MIEMBROS DE LA COMISIÓN:
GONZALO NAVARRO B.
IGNACIO SLATER M.

SANTIAGO DE CHILE

2026

Resumen

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

Quisque ullamcorper placerat ipsum. Cras nibh. Morbi vel justo vitae lacus tincidunt ultrices. Lorem ipsum dolor sit amet, consectetur adipiscing elit. In hac habitasse platea dictumst. Integer tempus convallis augue. Etiam facilisis. Nunc elementum fermentum wisi. Aenean placerat. Ut imperdiet, enim sed gravida sollicitudin, felis odio placerat quam, ac pulvinar elit purus eget enim. Nunc vitae tortor. Proin tempus nibh sit amet nisl. Vivamus quis tortor vitae risus porta vehicula.

Una dedicatoria corta.

Agradecimientos

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Tabla de Contenido

1. Introducción	1
1.1. Contexto	1
1.2. Problema y enfoque de solución	3
1.3. Objetivos	6
1.4. Contribuciones	6
2. Marco Teórico	8
2.1. Fundamentos de Haskell	8
2.2. Abstracciones funcionales	10
2.2.1. Functores	10
2.2.2. Aplicativos	11
2.2.3. Mónadas	13
2.3. Do-Notation	15
2.4. Mónadas Conmutativas	16
2.5. Mónadas de Probabilidad	17
2.6. ApplicativeDo	20
2.6.1. Etapa de Rearrange	20
2.6.2. Etapa de Desugaring	23
2.6.3. Aplicación del algoritmo	25
3. Problema	28
3.1. Limitación del orden sintáctico de ApplicativeDo	28

3.2. Evidencia del problema y oportunidad	29
3.3. Formulación del problema	31
4. Diseño de Solución	33
4.1. Flujo general de la extensión	33
4.2. Grafo de precedencia RAW	34
4.2.1. Modelo de dependencias	35
4.3. Generación de reordenamientos válidos	37
5. Implementación de la Solución	39
5.1. Flujo de compilación de GHC	39
5.2. Activación mediante CommutativeDo	41
5.3. Integración en el Renamer	42
5.4. Construcción del Grafo de Precedencia RAW	44
5.5. Generación de Reordenamientos Válidos	45
5.6. Generación y Selección de Planes	47
6. Validación	49
6.1. Metodología	49
6.2. Corpus de validación	53
6.2.1. Familias compartidas	53
6.2.2. Familias específicas	54
6.2.3. Control negativo con IO	55
6.3. Alcance y limitaciones de la validación	55
7. Resultados	57
7.1. Resultados generales	57
7.2. Casos destacables	58
7.2.1. Formas de dependencia	59
7.2.2. Dependencias introducidas por statements let	62

7.2.3.	Binders definidos mediante patrones	62
7.2.4.	Distribuciones con parámetros dependientes	63
7.2.5.	Condicionamiento de una distribución dependiente	64
7.2.6.	Identidades únicas frente a shadowing	65
7.3.	Selección por costo	65
7.3.1.	Todos los candidatos con el mismo costo	66
7.3.2.	Orden original no mínimo	67
7.3.3.	Mínimo único	67
7.3.4.	Varios candidatos mínimos	68
7.4.	Control negativo con IO	69
7.5.	Discusión	71
8.	Conclusión	73
8.1.	Resumen del Trabajo	73
8.2.	Cumplimiento de Objetivos	73
8.3.	Limitaciones	74
8.4.	Trabajo Futuro	75
8.5.	Cierre	75
	Bibliografía	78
	Apéndice A. Anexos Técnicos	79
A.1.	Reproducibilidad	79
A.2.	Pipeline de Validación	79
A.3.	Logs y Trazas	79
A.4.	Corpus	80
A.5.	Detalles de Implementación	80

Índice de Tablas

1.1. Probabilidad de aprobar el examen final según preparación y dificultad. . . .	3
1.2. Comparación de planes y costos estructurales para el ejemplo académico. . .	5
4.1. Ordenamientos topológicos del grafo de precedencia RAW de la Figura 1.1. .	38
6.1. Métricas extraídas de las trazas del Renamer.	52
6.2. Familias y variantes compartidas por los dos corpus.	54
6.3. Familias específicas de cada corpus.	54
7.1. Costos y cantidades de reordenamientos de la familia <code>cost-selection</code>	66
7.2. Salidas observables producidas por los dos órdenes del control negativo con IO.	70

Índice de Ilustraciones

1.1. Grafo de precedencia RAW del Código 1.2 dispuesto según el orden original [1,2,3,4].	4
1.2. Mismo grafo de precedencia RAW de la Figura 1.1 dispuesto según el reordenamiento válido de statements [1,3,2,4].	4
2.1. Etapa de <i>rearrange</i> propuesta por Marlow et al. [25, p. 95].	21
2.2. Etapa de <i>desugaring</i> propuesta por Marlow et al. [25, p. 96].	24
4.1. Flujo general de la extensión propuesta.	34
5.1. Flujo de renombrado y condiciones de activación de la ruta de reordenamiento de statements.	43
5.2. Árbol de llamadas para generar todos los reordenamientos válidos del programa probabilístico con Código 1.2. La raíz representa la invocación inicial y los otros 15 estados corresponden a llamadas recursivas.	47
6.1. Flujo de validación de la solución para responder PV1 y PV2	51
7.1. Grafo RAW del caso chain-no-reorder	59
7.2. Grafo RAW del caso diamond	60
7.3. Grafo RAW del caso no-deps	61
7.4. Grafo RAW del caso two-chains	61
7.5. Grafo RAW del caso bind-depends-on-let	62
7.6. Grafo RAW del caso tuple-pattern	63
7.7. Grafo RAW del caso choose-dependent	64
7.8. Grafo RAW del caso filter-bind	64

7.9. Grafo RAW del caso <code>read-after-rebind</code>	65
7.10. Grafo RAW del caso <code>all-same-cost</code>	66
7.11. Grafo RAW del caso <code>original-not-minimal</code>	67
7.12. Grafo RAW del caso <code>unique-minimum</code>	68
7.13. Grafo RAW del caso <code>many-minimums</code>	69
7.14. Grafo RAW del control con IO.	70

Capítulo 1

Introducción

1.1. Contexto

La programación probabilística es un paradigma que permite describir modelos con incertidumbre mediante programas que incorporan operaciones para extraer valores desde distribuciones de probabilidad y, en muchos lenguajes, condicionar esos valores mediante observaciones. A diferencia de un programa determinista, cuyo resultado queda determinado por su entrada, un programa probabilístico especifica implícitamente una distribución sobre sus posibles resultados. La inferencia probabilística consiste en obtener una representación de esa distribución o calcular propiedades de ella, como probabilidades, valores esperados o muestras [15].

Como lenguaje de modelado, la programación probabilística se ha aplicado en dominios como visión computacional, teoría de códigos, protocolos criptográficos, biología y análisis de confiabilidad [15]. Su relación con el aprendizaje automático y la inteligencia artificial es especialmente estrecha: los modelos probabilísticos permiten representar y manipular incertidumbre sobre datos, modelos y predicciones, mientras que los sistemas de programación probabilística permiten expresar esos modelos como procesos computacionales sobre los cuales aplicar inferencia bayesiana [9, 31].

El resultado observado al evaluar un programa probabilístico depende de la estrategia de interpretación utilizada. Bajo una interpretación por muestreo, cada ejecución concreta produce una muestra de la distribución inducida; por ejemplo, una ejecución del lanzamiento de una moneda retorna “cara” o “sello”. Una estrategia de inferencia exacta, en cambio, puede construir una representación explícita de la distribución completa. En ambos casos, el programa describe semánticamente la distribución sobre todos los resultados posibles, no solo una muestra particular [15, 6].

En lenguajes funcionales puros como **Haskell**, es posible encapsular este tipo de cálculos en tipos de datos y componerlos mediante mónadas de probabilidad. Para distribuciones discretas y finitas, una representación monádica puede construir explícitamente los resultados posibles junto con sus probabilidades, lo que permite realizar inferencia exacta sobre la distribución inducida por el programa [6]. El mismo enfoque monádico también puede servir

como interfaz para métodos de inferencia más generales, sin restringirse a la enumeración exhaustiva [31].

Estas representaciones permiten realizar consultas y operaciones sobre la distribución de salida de un programa. Por ejemplo, la distribución conjunta asociada al lanzamiento de una moneda seguido del lanzamiento de un dado puede expresarse mediante la *do-notation* de Haskell [6]:

```
flipCoinRollDice = do
  c <- uniform [Head, Tail]
  d <- uniform [1 .. 6]
  return (c, d)
```

Código 1.1: Programa probabilístico que modela el lanzamiento conjunto de una moneda y un dado de seis caras mediante la *do-notation* de Haskell.

En cada resultado posible del código anterior, la variable `c` contiene un resultado del lanzamiento de la moneda y la variable `d` contiene una cara del dado. Como la segunda elección no depende del resultado de la primera, `return (c, d)` construye la distribución conjunta de ambos experimentos. Su soporte contiene los doce pares obtenidos al combinar los dos resultados de la moneda con las seis caras del dado y, bajo las elecciones uniformes del ejemplo, cada par tiene probabilidad $1/12$ [6].

Tal como se aprecia en el ejemplo, la *do-notation* es especialmente conveniente para programas probabilísticos: un statement de enlace puede introducir una variable aleatoria o un resultado intermedio, y los statements posteriores pueden depender de los valores que ya han sido definidos. En el desazucarado monádico convencional de Haskell, un statement de la forma `p <- e` se traduce internamente mediante el operador *bind* (`>>=`) [22, sección 3.14], permitiendo que la forma de un cómputo posterior dependa del valor producido por uno anterior. Como consecuencia del encadenamiento mediante *bind*, la ejecución resultante del bloque es inherentemente secuencial y sigue el orden de sus statements. Esta capacidad de encadenar cómputos dependientes es fundamental para modelar efectos como estado, entrada/salida, fallas, no determinismo o distribuciones probabilísticas [33, 6].

La secuencia monádica es correcta para el caso general, pero puede ocultar independencia entre subcómputos. Esta tensión fue abordada en GHC mediante la extensión **ApplicativeDo**, propuesta por Marlow et al. [25]. La extensión analiza las dependencias de un bloque escrito con *do-notation* y, cuando es posible, construye un plan de ejecución mixto que utiliza operadores aplicativos como (`<$>`) y (`<*>`) junto con las operaciones monádicas que todavía sean necesarias [25, 13]. De este modo, el programador conserva la sintaxis habitual y el plan resultante puede exponer independencia que ciertas instancias aplicativos aprovechan mediante paralelismo o agrupación de operaciones.

Sin embargo, el algoritmo estándar de **ApplicativeDo** está diseñado para mónadas generales, sin asumir que sus efectos sean conmutativos. Por esta razón, para obtener el nuevo plan de ejecución agrupa statements compatibles pero no cambia su orden relativo: una permutación de estos podría alterar efectos observables, mientras que el intercambio de cómputos

independientes solo es válido bajo una hipótesis adicional de conmutatividad [25, pp. 94–95]. La subsección siguiente muestra, mediante un programa probabilístico concreto, cómo esta restricción puede impedir que el algoritmo de `ApplicativeDo` genere un plan de ejecución de menor costo.

1.2. Problema y enfoque de solución

Considérese un modelo probabilístico sencillo sobre el desempeño de un estudiante. El modelo representa el nivel de preparación del estudiante, el resultado de un quiz, la dificultad de un examen final y el resultado de ese examen. La preparación y la dificultad se modelan como elecciones equiprobables e independientes. En cambio, los resultados de las evaluaciones se modelan mediante distribuciones condicionadas: la probabilidad de aprobar el quiz es $1/4$ para una preparación baja y $3/4$ para una preparación alta; la probabilidad de aprobar el examen final se resume en la Tabla 1.1.

Preparación	Dificultad	Probabilidad de aprobar
Low	Easy	$1/2$
Low	Hard	$1/4$
High	Easy	$3/4$
High	Hard	$1/2$

Tabla 1.1: Probabilidad de aprobar el examen final según preparación y dificultad.

Las distribuciones condicionales se implementan mediante las funciones `quizGiven` y `examGiven`. El cómputo conjunto se escribe con la *do-notation* de la siguiente forma:

```
studentResults = do
  preparation <- Dist.uniform [Low, High]
  quiz <- quizGiven preparation
  difficulty <- Dist.uniform [Easy, Hard]
  finalExam <- examGiven preparation difficulty
  return (preparation, quiz, difficulty, finalExam)
```

Código 1.2: Modelo probabilístico del desempeño de un estudiante.

Para analizar el bloque, se numeran los cuatro statements previos al `return` como `s1`, `s2`, `s3` y `s4`. El statement `s2` usa la preparación definida por `s1`; el statement `s4` usa tanto la preparación de `s1` como la dificultad definida por `s3`. No existe, en cambio, una dependencia entre el resultado del quiz en `s2` y la dificultad en `s3`. Estas relaciones corresponden a dependencias de tipo *Read-after-Write* (RAW), en las que un statement usa un valor definido por otro con anterioridad. Para representar las precedencias que imponen estas dependencias, se utiliza un *grafo de precedencia RAW*. En él, cada statement constituye un vértice y cada dependencia se expresa mediante una arista dirigida desde el statement que define un valor hacia aquel que lo usa. En este caso, para el programa del Código 1.2 las aristas son `s1 → s2`, `s1 → s4` y `s3 → s4`.

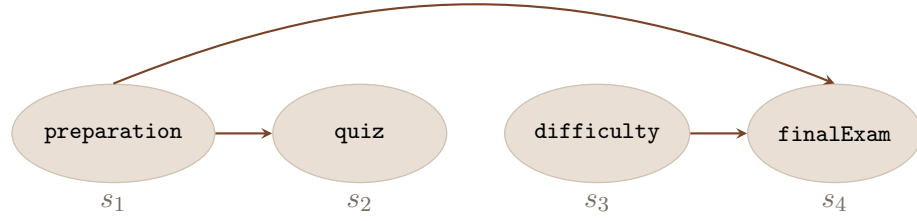


Figura 1.1: Grafo de precedencia RAW del Código 1.2 dispuesto según el orden original $[1, 2, 3, 4]$.

En el grafo de precedencia RAW anterior, la posición horizontal de los nodos representa el orden de los statements: un nodo ubicado a la izquierda aparece antes que uno ubicado a la derecha. Una disposición respeta las dependencias del programa solo si todas las aristas avanzan de izquierda a derecha, pues cada arista exige que el statement que produce un valor aparezca antes del statement que lo usa. Por consiguiente, una arista dirigida hacia la izquierda indicaría que ese orden invierte y rompe la dependencia.

El grafo no impone un orden relativo entre s_2 y s_3 , de modo que pueden intercambiarse para obtener el orden $[1, 3, 2, 4]$. Este orden conserva s_1 antes de s_2 y s_4 , y s_3 antes de s_4 ; por ello, las tres aristas siguen avanzando hacia la derecha, como muestra la Figura 1.2. Lo anterior se considera como un *reordenamiento válido* de los statements, más formalmente este concepto se define como toda secuencia reordenada de los statements que preserve las dependencias locales del bloque `do` original. Bajo una hipótesis explícita de conmutatividad, los reordenamientos válidos se consideran además semánticamente equivalentes al orden original.

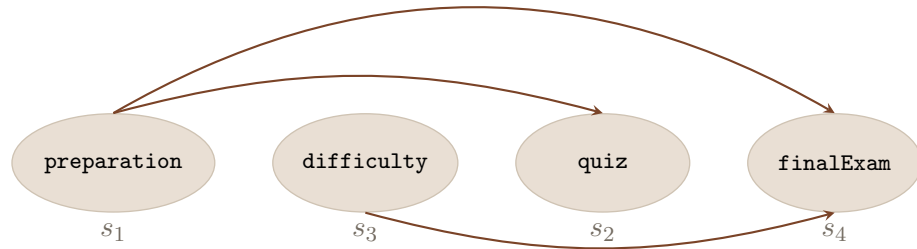


Figura 1.2: Mismo grafo de precedencia RAW de la Figura 1.1 dispuesto según el reordenamiento válido de statements $[1, 3, 2, 4]$.

La validez del reordenamiento $[1, 3, 2, 4]$ asegura que el intercambio preserve las dependencias del bloque, pero no permite determinar si el nuevo orden ofrece alguna ventaja frente al original. Como `ApplicativeDo` conserva el orden de los statements que recibe, una disposición distinta puede cambiar las agrupaciones aplicativas que logra construir y, por tanto, la estructura del cómputo resultante. Para representar estas estructuras y comparar posteriormente sus costos se utiliza un *plan de ejecución*, una representación abstracta que indica qué partes del programa permanecen secuenciadas y cuáles pueden agruparse aplicativamente. Este plan no describe una planificación concreta de hilos ni tiempos reales de ejecución.

Un plan de ejecución se describe mediante dos operadores conceptuales. El símbolo $;$ representa composición secuencial: el plan de la derecha comienza una vez finalizado el plan de la izquierda. El símbolo $|$ representa una agrupación aplicativa de planes independientes y expresa paralelismo potencial, no necesariamente una ejecución multihilo. Sobre esta notación se adopta el siguiente modelo de costos estructural simplificado. Sea C la función que asigna un costo a cada plan de ejecución. Si s_i es un statement elemental y P , Q y P_i representan planes o subplanes, entonces:

$$C(s_i) = 1, \quad C(P; Q) = C(P) + C(Q), \quad C(P_1 | \dots | P_k) = \max_{1 \leq i \leq k} C(P_i).$$

Expresión 1.1: Modelo de costos estructural simplificado.

Estas reglas asignan un costo unitario a cada statement elemental, de modo que $C(s_i) = 1$. Cuando dos planes se componen secuencialmente mediante $;$ sus costos se suman debido a la secuencialidad del mismo. En una agrupación aplicativa $P_1 | \dots | P_k$, en cambio, el modelo toma el máximo de los costos de sus ramas independientes; este valor representa el camino crítico de la agrupación. Por consiguiente, el costo mide la estructura del plan y no el tiempo real de ejecución.

A partir de estas reglas, la Tabla 1.2 presenta los planes de ejecución y sus costos para el programa del Código 1.2 en tres variantes: una ejecución base completamente secuencial, sin agrupaciones aplicativas; **ApplicativeDo** aplicado al orden original; y **ApplicativeDo** aplicado al reordenamiento válido $[1, 3, 2, 4]$ mostrado en la Figura 1.2.

Variante	Plan de ejecución	Costo
Secuencial	$s_1; s_2; s_3; s_4$	4
ApplicativeDo	$s_1; (s_2 (s_3; s_4))$	3
Reordenamiento válido	$(s_1 s_3); (s_2 s_4)$	2

Tabla 1.2: Comparación de planes y costos estructurales para el ejemplo académico.

La comparación muestra que **ApplicativeDo** mejora el plan base completamente secuencial: al aplicarse sobre el orden original, reduce su costo de 4 a 3 mediante una agrupación aplicativa. Sin embargo, para mantener la corrección sobre mónadas arbitrarias, este conserva el orden relativo de los statements y, por ello, no es capaz de generar el plan de ejecución con costo 2. Este último requiere intercambiar s_2 y s_3 , como ocurre en el reordenamiento válido $[1, 3, 2, 4]$. La preservación del orden es necesaria en el caso general, pues los efectos observables de una mónada pueden depender de él, pero puede resultar demasiado conservadora cuando el bloque se trata bajo una hipótesis explícita de conmutatividad.

Esta limitación motiva la extensión de **ApplicativeDo** estudiada en esta memoria. Cuando un bloque **do** se marca explícitamente bajo una hipótesis de conmutatividad, la extensión implementada considera todos los reordenamientos válidos de sus statements antes de construir el plan de ejecución. El algoritmo existente de **ApplicativeDo** se aplica a cada reordenamiento válido y, entre los planes resultantes, se selecciona el de menor costo según el

modelo estructural adoptado. De este modo, se amplía el espacio de búsqueda sin reemplazar el algoritmo ni modificar el tratamiento de los bloques `do` no marcados.

Al aplicar esta extensión de **ApplicativeDo** al modelo académico del Código 1.2, se generaron cinco reordenamientos válidos. El reordenamiento válido `[1,3,2,4]` correspondió al primer candidato de costo mínimo y produjo el plan de costo 2 presentado en la Tabla 1.2. Para validar experimentalmente la preservación del resultado, se ejecutaron el programa original, la variante estándar de **ApplicativeDo**, el candidato de costo mínimo seleccionado automáticamente y cada uno de los cinco reordenamientos mediante la instrumentación descrita en el capítulo de Validación. Todas las variantes produjeron exactamente la misma distribución normalizada, lo que entrega evidencia de preservación semántica para este ejemplo. Los capítulos siguientes desarrollan la formulación del problema, el diseño y la implementación de este proceso y su evaluación experimental.

1.3. Objetivos

El objetivo general de esta memoria es ampliar experimentalmente el espacio de planes de ejecución que **ApplicativeDo** puede evaluar en **GHC**, considerando reordenamientos válidos de statements dentro de bloques `do` marcados explícitamente bajo una hipótesis de conmutatividad. Con ello, se busca seleccionar, según el modelo de costos adoptado, un plan de menor costo cuando alguno de estos reordenamientos válidos lo permita.

Los objetivos específicos son los siguientes:

1. Caracterizar los reordenamientos válidos de los statements de un bloque `do` según las dependencias locales que deben preservarse.
2. Ampliar el conjunto de planes de ejecución que **ApplicativeDo** puede considerar para bloques `do` bajo una hipótesis explícita de conmutatividad, de modo que se pueda seleccionar un plan de ejecución con un costo estructural mínimo entre los obtenidos a partir de reordenamientos válidos.
3. Comprobar experimentalmente, en los casos conmutativos evaluados, que los reordenamientos válidos preservan el resultado observable del programa y que la selección realizada corresponde a un plan de ejecución con costo estructural mínimo.

1.4. Contribuciones

Las principales contribuciones de esta memoria son:

- Un mecanismo para generar reordenamientos válidos de statements, definidos por la preservación de las dependencias locales del programa y considerados semánticamente equivalentes al orden original bajo una hipótesis explícita de conmutatividad.

- Una extensión experimental de **GHC** que integra este mecanismo con **ApplicativeDo**, amplía el espacio de planes de ejecución considerado por el compilador y selecciona una alternativa de costo mínimo según el modelo estructural adoptado.
- Una validación experimental reproducible que comprueba, bajo la hipótesis explícita de conmutatividad y en los casos evaluados, que todos los reordenamientos válidos generados producen el mismo resultado observable que el programa original y que la selección automática corresponde a un plan de costo estructural mínimo.

Capítulo 2

Marco Teórico

Este capítulo presenta los fundamentos conceptuales necesarios para comprender tanto el problema abordado en esta memoria como la solución desarrollada. En primer lugar, se introducen los fundamentos de **Haskell** necesarios para interpretar los ejemplos y las firmas de tipos utilizadas a lo largo del informe. Sobre esta base, también se estudian sus principales abstracciones funcionales: funtores, aplicativos y mónadas. Estas clases forman una jerarquía que permite estructurar cómputos dentro de un contexto y se distinguen por el grado de dependencia que pueden expresar entre las distintas partes de un programa.

A continuación, se profundiza en la *do-notation*, explicando su relación con la composición monádica y cómo sus statements de enlace se traducen, en su forma convencional, mediante el operador *bind* ($>>=$). Esto es fundamental para entender por qué **ApplicativeDo** puede reemplazar parte de una expresión monádica por una expresión aplicativa [25, 13].

Sobre esta base, se introduce la noción de mónada conmutativa, cuya propiedad relevante permite intercambiar ciertos cómputos independientes sin modificar la semántica del programa [20, 17]. Posteriormente, se presentan las mónadas de probabilidad como un caso canónico de esta propiedad y como el principal dominio de aplicación de la memoria.

Finalmente, se estudia la extensión **ApplicativeDo**, describiendo la transformación que realiza sobre bloques **do**, la forma en que identifica dependencias entre statements y la generación de expresiones aplicativas que pueden exponer una mayor estructura paralelizable. Este recorrido permite establecer los conceptos necesarios para analizar posteriormente las limitaciones del mecanismo actual y la extensión experimental propuesta.

2.1. Fundamentos de Haskell

Haskell es un lenguaje de programación funcional puro, de propósito general y con tipado estático. En este lenguaje, las funciones desempeñan un papel central y son tratadas como valores de primera clase: pueden recibirse como argumentos, retornarse como resultados y almacenarse en estructuras de datos. Esta característica permite que los programas no solo operen con valores como números o listas, sino también con las funciones que transforman

dichos valores [22, capítulo 1 y sección 6.1.6].

La pureza implica que la evaluación de una función depende de sus argumentos y no produce efectos secundarios implícitos. Los efectos se representan de manera explícita mediante tipos y abstracciones que determinan cómo se componen los cálculos. Esta separación será relevante al introducir mónadas y analizar el orden de los efectos expresados mediante bloques `do` [22, capítulo 7] [33].

Además, las funciones de **Haskell** se encuentran curricadas. Esto significa que una función que conceptualmente recibe varios argumentos se representa como una sucesión de funciones que reciben un argumento a la vez. Por esta razón, las flechas de una firma de tipos se asocian hacia la derecha; por ejemplo, el tipo `a -> b -> c` se interpreta como `a -> (b -> c)`. Una función de este tipo recibe un valor de tipo `a` y retorna una nueva función de tipo `b -> c` [22, secciones 3.3 y 4.1.2].

Este comportamiento puede observarse en el operador binario `(+)`, cuya firma de tipos se muestra en el Código 2.1 [22, sección 6.4 y figura 6.2].

```
(+) :: Num a => a -> a -> a
```

Código 2.1: Firma de tipos del operador de suma `(+)`.

En esta firma, `a` es una variable de tipo y la restricción `Num a` establece que debe elegirse un tipo perteneciente a la clase `Num`. Si se aplica `(+)` únicamente al valor `5`, se fija su primer argumento y se obtiene una nueva función que todavía espera el argumento restante. Esta operación se denomina *aplicación parcial*. En este caso, la expresión `(+) 5` produce la función unaria `(5+)` de tipo `Num a => a -> a` [22, secciones 3.5, 4.1.3 y 6.4].

Los tipos también pueden estar parametrizados. Por ejemplo, **Maybe** es un constructor de tipos que recibe un tipo `a` y produce el tipo `Maybe a`. Sus constructores de datos representan dos posibilidades: **Just** contiene un valor de tipo `a` y **Nothing** representa su ausencia. Esta distinción entre un constructor de tipos y los tipos concretos que produce permite definir comportamientos comunes para toda la familia **Maybe** [22, secciones 4.1.2 y 6.1.8; capítulo 21].

Una clase de tipos especifica un conjunto de operaciones que distintos tipos o constructores de tipos pueden implementar. Cada implementación concreta se declara mediante una instancia [22, secciones 4.1 y 4.3]. La restricción `Num a` del ejemplo anterior indica precisamente que debe existir una instancia de `Num` para el tipo elegido. De manera análoga, **Functor**, **Applicative** y **Monad** son clases de tipos con operaciones e instancias propias. **Maybe** posee instancias para las tres y, por ello, se utilizará como ejemplo común para mostrar cómo un mismo contexto puede transformar valores, combinar cálculos y expresar dependencias entre sus resultados [34, 32].

2.2. Abstracciones funcionales

Las funciones de primera clase, la currificación, la aplicación parcial y las clases de tipos proporcionan la base para las abstracciones estudiadas en esta sección. **Functor**, **Applicative** y **Monad** forman una jerarquía que se distingue por el grado de dependencia que permiten expresar entre las partes de un programa. Un functor permite transformar el resultado de un cómputo; un aplicativo permite combinar cómputos cuya estructura está determinada de antemano; y una mónada permite que un cómputo posterior dependa del valor producido por uno anterior [33, 27, 34].

2.2.1. Functores

En Haskell, **Functor** es una clase de tipos cuyas instancias se definen para constructores de tipos, como las listas `[]` o `Maybe`. Su operación principal es `fmap`, que permite aplicar una función a los valores asociados a una estructura y obtener un resultado que conserva el mismo constructor de tipos. La definición simplificada de la clase se muestra en el Código 2.2:

```
class Functor f where
    fmap :: (a -> b) -> f a -> f b
```

Código 2.2: Definición simplificada de la clase **Functor**.

En esta firma, `f` representa un constructor de tipos, mientras que `a` y `b` representan los tipos de los valores antes y después de aplicar la función. Así, `fmap` recibe una función de tipo `a -> b` y transforma un valor de tipo `f a` en uno de tipo `f b`, manteniendo el contexto representado por `f`.

En instancias concretas como las listas y `Maybe`, esta operación puede entenderse intuitivamente mediante la metáfora de un contenedor: `fmap` transforma los valores contenidos sin modificar la estructura que los organiza. Bajo esta interpretación pedagógica, puede imaginarse que la operación accede a los valores, aplica una función sobre ellos y vuelve a representarlos dentro del mismo contexto. Sin embargo, esta metáfora no constituye la definición general de un functor.

Además, toda instancia válida de **Functor** debe respetar las leyes de identidad y composición. Estas leyes garantizan que la aplicación de `fmap` preserve de manera coherente la estructura asociada al constructor de tipos [34].

Los ejemplos clásicos de functores incluyen las listas y los árboles, pues `fmap` transforma sus elementos de manera uniforme mientras mantiene su estructura subyacente.

Como se mencionó anteriormente, otro ejemplo relevante es `Maybe`. El tipo `Maybe a` representa valores opcionales: un valor disponible se expresa mediante el constructor de datos `Just`, mientras que la ausencia de un valor se representa mediante `Nothing`. Al utilizar `fmap` sobre este tipo, la función recibida se aplica al valor contenido en `Just`, mientras que `Nothing`

permanece sin cambios.

En la implementación de **base** incluida con GHC, este comportamiento se define mediante la instancia de **Functor** para el constructor de tipos **Maybe** mostrada en el Código 2.3 [32]:

```
instance Functor Maybe where
  fmap _ Nothing = Nothing
  fmap f (Just a) = Just (f a)
```

Código 2.3: Instancia de **Functor** para **Maybe**.

La primera ecuación establece que, cuando el valor recibido es **Nothing**, no existe ningún elemento sobre el cual aplicar la función y, por tanto, el resultado permanece como **Nothing**. El guion bajo indica que la función recibida no necesita utilizarse en este caso. En cambio, cuando el valor tiene la forma **Just a**, la segunda ecuación aplica la función **f** al elemento **a** y vuelve a encapsular el resultado mediante el constructor **Just**. Ambos comportamientos se ilustran en el Código 2.4:

```
> fmap (+1) (Just 2)
Just 3
> fmap (+1) Nothing
Nothing
```

Código 2.4: Aplicación de **fmap** sobre valores de tipo **Maybe**.

2.2.2. Aplicativos

En Haskell, **Applicative** es una clase de tipos definida para constructores de tipos que ya poseen una instancia de **Functor**. Sus operaciones principales son **pure** y **(<*>)**: la primera introduce un valor en un contexto, mientras que la segunda aplica una función contextualizada a un valor dentro de ese mismo contexto. De esta forma, la clase caracteriza un estilo aplicativo de programación, menos expresivo que el monádico y, por tanto, aplicable a una mayor variedad de estructuras [27]. Para ilustrar este comportamiento, se utilizará nuevamente el constructor de tipos **Maybe** y el operador binario **(+)**.

Un constructor de tipos que posee una instancia de **Applicative** se denomina aplicativo. Considerando **Maybe**, para sumar **Just 2** y **Just 3**, la función **(+)** se introduce primero en el contexto mediante **pure**. La primera aplicación de **(<*>)** produce la función unaria contextualizada **Just ((+) 2)**, de tipo **Maybe (Int -> Int)**. La segunda aplicación combina esta función con **Just 3** y produce el resultado **Just 5**.

Cada instancia de **Applicative** define cómo se comportan estas operaciones para un constructor de tipos concreto. En la instancia de **Maybe**, si alguno de los cómputos asociados a los sumandos produce **Nothing**, esta ausencia se propaga al resultado. De esta forma, el comportamiento de los valores opcionales queda definido una sola vez en la instancia, evitando

repetir ajustes de patrones cada vez que se combinan operaciones que pueden fallar.

Formalmente, la clase `Applicative` extiende `Functor` y declara las operaciones `pure` y `(<*>)` mediante la definición simplificada del Código 2.5:

```
class Functor f => Applicative f where
  pure :: a -> f a
  <*>  :: f (a-> b) -> f a -> f b
```

Código 2.5: Definición simplificada de la clase `Applicative`.

En particular, la instancia `Applicative Maybe` define los operadores `pure` y `<*>` de acuerdo con el comportamiento de los valores opcionales: permite introducir valores en el contexto y combinar operaciones que pueden fallar. Una definición de esta instancia se presenta en el Código 2.6:

```
instance Applicative Maybe where
  pure = Just
  Nothing <*> _ = Nothing
  _ <*> Nothing = Nothing
  (Just f) <*> (Just x) = Just (f x)
```

Código 2.6: Instancia de `Applicative` para `Maybe`.

Nótese que, en caso de operar con algún fallo de cómputo o ausencia de algún valor, cuya abstracción equivale al constructor `Nothing`, las siguientes aplicaciones retornarán a su vez un elemento `Nothing`, propagando así el comportamiento definido por la instancia `Applicative Maybe`. En caso contrario, si ambos operandos se encuentran definidos, es decir, son elementos encapsulados en el constructor `Just`, se procede con la aplicación del primero sobre el segundo, para finalmente empaquetar el elemento resultante dentro del mismo constructor `Just`.

Volviendo al ejemplo de aplicación de la suma (+) sobre elementos de tipo `Maybe Int`, se requiere que esta se encuentre encapsulada dentro del constructor `Just`. El operador `<*>` espera que su operando izquierdo sea una función dentro del contexto. Por ello, `pure (+)` introduce la función de suma en `Maybe`, produciendo el valor contextualizado requerido. La expresión aplicativa concreta se muestra en el Código 2.7:

```
pure (+) <*> Just 2 <*> Just 3
```

Código 2.7: Suma aplicativa de dos valores de tipo `Maybe Int`.

Como `Functor` es una superclase de `Applicative`, todo constructor de tipos con una instancia de `Applicative` también dispone de `fmap`. El operador `(<$>)` corresponde a su forma infija, por lo que se cumple la igualdad `f <$> m = fmap f m`. Además, la coherencia entre ambas clases establece que `fmap f m = pure f <*> m`. En consecuencia, ambas expre-

siones pueden relacionarse mediante $f \text{ <\$> } m = \text{fmap } f \ m = \text{pure } f \text{ <*> } m$. De este modo, el ejemplo anterior puede escribirse con la sintaxis más directa del Código 2.8:

```
(+) <\$> Just 2 <*> Just 3
```

Código 2.8: Forma infija de la suma aplicativa sobre `Maybe`.

Finalmente, nótese que en una expresión aplicativa, la estructura del cómputo queda fijada de antemano: cada argumento se evalúa en su propio contexto y luego se combinan sus resultados mediante la aplicación de una función pura. Esta característica hace explícita la ausencia de dependencias de datos entre los subcómputos asociados a los argumentos, pues ninguno puede usar el resultado del otro para determinar su propia forma. Esto no implica que sus efectos sean conmutativos, aunque puede habilitar una evaluación más paralelizable cuando la implementación lo permite [27].

Esta misma restricción implica que, si se quisiera que el segundo argumento dependa del valor producido por el primero, el estilo aplicativo resulta insuficiente, ya que la forma del cómputo no puede adaptarse dinámicamente a resultados intermedios. Por otro lado, la abstracción monádica proporciona el operador *bind* (`>>=`), que permite construir el segundo cómputo a partir del resultado del primero. Esta capacidad de expresar dependencias entre resultados intermedios motiva la introducción de la clase `Monad` en `Haskell` [33].

2.2.3. Mónadas

En `Haskell`, `Monad` es una clase de tipos cuyas instancias definen cómo secuenciar cómputos dentro de un mismo contexto cuando un cómputo posterior puede depender del valor producido por uno anterior [33]. Como `Applicative` es una superclase de `Monad`, todo constructor de tipos que posea una instancia de `Monad` también debe poseer una instancia de `Applicative`. El Código 2.9 muestra una definición simplificada con las operaciones relevantes:

```
class Applicative m => Monad m where
  (>>=) :: m a -> (a -> m b) -> m b
  return :: a -> m a
```

Código 2.9: Definición simplificada de la clase `Monad`.

Para interpretar la firma de (`>>=`), consideraremos un cómputo `ma :: m a` y una función `k :: a -> m b`. La expresión `ma >>= k` compone el primer cómputo con la función `k` de acuerdo con el comportamiento definido por la instancia, por ende entrega su resultado de tipo `a` a la función `k` y obtiene un nuevo cómputo de tipo `m b`. De esta manera, la función `k` puede utilizar el resultado anterior para determinar la forma del cómputo siguiente.

La operación `return` introduce un valor en el contexto monádico y, en la jerarquía actual de clases de tipos, coincide con `pure`. Su nombre no representa una transferencia de control como en lenguajes imperativos, sino la construcción de un valor de tipo `m a` a partir de uno

de tipo `a`.

Al encadenar varias aplicaciones de (`>>=`), las funciones entregadas al operador pueden escribirse como funciones anónimas λ . Estas permiten asignar nombres a los resultados intermedios y utilizarlos en los cálculos posteriores. Para ilustrar este comportamiento, se retomará el ejemplo de la suma con `Maybe`.

En este caso, la instancia `Monad Maybe` define el comportamiento del operador `>>=` para los valores opcionales. Las operaciones relevantes se presentan en el Código 2.10:

```
instance Monad Maybe where
  return x = Just x
  Nothing >>= _ = Nothing
  (Just x) >>= f = f x
```

Código 2.10: Instancia de `Monad` para `Maybe`.

Cuando el primer cálculo produce `Nothing`, la ausencia de valor se propaga como resultado. En cambio, cuando produce un valor de la forma `Just x`, el elemento `x` se entrega como argumento a la función ubicada a la derecha de `>>=`. En este caso, la función que se ejecuta debe respetar la firma de tipos y retornar un nuevo cálculo de tipo `m b`. El Código 2.11 presenta la aplicación de la función `(+)` mediante notación monádica:

```
Just 3 >>= (\x -> Just 2 >>= (\y -> return ((+) x y)))
```

Código 2.11: Suma monádica de dos valores de tipo `Maybe`.

Por tanto, al ejecutar el operador `>>=`, se desempaqueta el número 3 y se aplica la función anónima de la derecha sobre este valor, reemplazando todas las apariciones del identificador `x` por 3. A continuación, se realiza el mismo proceso con el cálculo `Just 2`, resultando finalmente en la expresión `((+) 3 2)` que será empaquetada gracias a la instrucción `return` retornando el valor contextualizado `Just 5`.

Cabe recalcar que ahora el identificador `x` queda dentro del alcance de la función que construye el segundo cálculo. Por ello, la expresión asociada a `Just 2` puede reemplazarse por otra que utilice `x`. El Código 2.12 muestra un programa que utiliza esta nueva expresividad de las mónadas:

```
Just 3 >>= (\x -> Just ((* 2 x) >>= (\y -> return ((+) x y)))
```

Código 2.12: Cálculo monádico con dependencia entre resultados intermedios.

Del programa anterior, el primer cálculo desempaqueta `Just 3` y reemplaza este elemento en las apariciones del identificador `x`. Luego, el cálculo posterior `Just ((* 2 x)` es capaz de considerar el valor numérico de `x` retornando `Just 6`, luego se desempaqueta su resultado y este reemplaza las apariciones del identificador `y`. Quedando como cálculo

final la operación `((+) 3 6)` cuyo resultado termina siendo empaquetado por la instrucción `return` como `Just 9`. Este flujo de ejecución permite una mayor expresividad enriqueciendo el programa. De esta manera la notación monádica termina siendo algo característico y central en `Haskell`.

Aunque esta composición permita expresar dependencias de forma correcta y versátil, la acumulación de funciones anónimas dificulta su lectura. La *do-notation*, presentada a continuación, proporciona una sintaxis más clara para escribir la misma secuencia monádica.

2.3. Do-Notation

La *do-notation* es una sintaxis de `Haskell` para escribir cálculos monádicos de forma secuencial. Un bloque `do` contiene statements que pueden vincular resultados intermedios y utilizarlos en los statements posteriores. Esta notación no define una forma nueva de composición, sino que proporciona una escritura más directa para expresiones construidas mediante las operaciones monádicas como puede ser el operador *bind* (`>>=`), (`>>`) y *fail*, además de contemplar una regla propia para declaraciones `let` [22, sección 3.14].

Para observar la motivación de esta sintaxis, se retomará la suma de dos valores de tipo `Maybe` presentada mediante el operador *bind* en el Código 2.11. Aunque la expresión es compacta, la acumulación de operadores, funciones anónimas y paréntesis dificulta distinguir su estructura a simple vista.

El Código 2.13 distribuye la misma expresión en varias líneas, sin cambiar el orden ni la agrupación de sus operaciones, para mostrar con mayor claridad el flujo de definición y composición de los cálculos:

```
Just 3 >>= (\x ->
  Just 2 >>= (\y ->
    return ((+) x y)))
```

Código 2.13: Forma multilínea de la suma monádica sobre `Maybe`.

La indentación permite reconocer que la composición con `Just 2` se encuentra dentro de la función que introduce `x`, mientras que `return` cierra la composición utilizando los identificadores `x` e `y`. Este cambio es únicamente visual: la expresión conserva la semántica del Código 2.11 y produce `Just 5`.

La *do-notation* permite expresar este mismo flujo sin escribir de forma explícita las funciones anónimas ni las aplicaciones sucesivas de (`>>=`). En un statement de la forma `p <- e`, la expresión `e` representa el cómputo y el patrón `p` vincula su resultado para que pueda utilizarse en los statements posteriores. El Código 2.14 presenta la escritura equivalente:

```
do
  x <- Just 3
  y <- Just 2
  return ((+) x y)
```

Código 2.14: Suma monádica sobre **Maybe** escrita mediante *do-notation*.

En el Código 2.14, el primer statement corresponde al cómputo entregado al ($\gg=$) exterior y vincula su resultado con **x**; el segundo corresponde al ($\gg=$) interior y vincula su resultado con **y**; el último construye el resultado del bloque. Al aplicar las reglas convencionales de traducción de la *do-notation*, este bloque recupera la composición anidada del Código 2.13 y, en última instancia, la expresión compacta del Código 2.11 [22, sección 3.14].

Aunque el bloque se escribe como una secuencia, en este ejemplo la expresión **Just 2** no utiliza el valor de **x**; por ello, entre los dos primeros statements no existe una dependencia de datos local. Ambos resultados sí son utilizados por el statement final. Esta distinción entre orden sintáctico y dependencias de datos será relevante más adelante: la notación hace más legible la composición, pero no autoriza por sí sola a intercambiar cómputos. Las condiciones que permiten transformar o reordenar partes de esta secuencia se estudiarán al introducir las mónadas conmutativas y **ApplicativeDo**.

2.4. Mónadas Conmutativas

La sección anterior mostró que la ausencia de una dependencia de datos entre dos statements permite formular dos órdenes distintos de estos mismos, pero no basta por sí sola para asegurar que el intercambio conserve el significado del programa. Las mónadas conmutativas proporcionan precisamente la propiedad semántica necesaria para justificarlo. En términos operacionales, esto significa que, dados dos cómputos monádicos independientes, ejecutar el primero y luego el segundo es semánticamente equivalente a ejecutarlos en el orden contrario, siempre que se conserven las asociaciones entre sus resultados.

Kock formaliza esta propiedad mediante dos mapas de Fubini, correspondientes a las dos formas de combinar un par de cómputos al efectuar primero uno u otro. Una mónada es conmutativa cuando ambos mapas coinciden; en ese caso, el orden elegido para realizar la combinación no modifica su significado [20, secciones 5 y 9].

La mónada **Maybe** permite ilustrar esta propiedad mediante el mismo ejemplo de suma utilizado en las secciones anteriores. El Código 2.15 presenta el bloque original y una variante en la que se intercambian los dos primeros statements:

<pre> sumaOriginal = do x <- Just 3 y <- Just 2 return ((+) x y) </pre>	$\Longleftrightarrow$	<pre> sumaReordenada = do y <- Just 2 x <- Just 3 return ((+) x y) </pre>
---	-----------------------	---

Código 2.15: Suma monádica sobre **Maybe** en el orden original y según un reordenamiento válido.

En `sumaOriginal`, el primer statement vincula el valor 3 con `x` y el segundo vincula el valor 2 con `y`. En `sumaReordenada` se invierte el orden de ejecución, pero se mantienen las mismas asociaciones: `x` continúa vinculado con 3 e `y` con 2. Como la expresión final tampoco cambia, ambos bloques producen `Just 5`.

Esta equivalencia no se debe a que la suma sea una operación conmutativa. El intercambio de statements no modifica el orden de los argumentos de `((+) x y)`, sino solamente el orden en que se obtienen los valores asociados a `x` e `y`. La propiedad relevante pertenece a la mónada: bajo la semántica total de valores opcionales, si ambos cómputos producen un valor de la forma `Just`, estos valores se conservan en cualquiera de los dos órdenes; si alguno produce `Nothing`, ambos órdenes producen `Nothing`. Esta caracterización no considera valores divergentes, como `undefined`, ni efectos externos.

Por lo tanto, conviene distinguir independencia estructural de conmutatividad. La expresión `Just 2` no utiliza `x`, de modo que intercambiar ambos statements conserva su alcance y sus dependencias de datos. Sin embargo, esta ausencia de dependencia no bastaría para afirmar conmutatividad en una mónada arbitraria: dos acciones de tipo `IO` podrían no utilizar sus resultados mutuamente y aun así imprimir mensajes en órdenes diferentes. La independencia permite formular ambos órdenes, mientras que la conmutatividad garantiza que el cambio de orden no altera los efectos observables [25, p. 94].

La noción también aparece de manera natural en el estudio de distribuciones. Kock presenta las mónadas conmutativas como una teoría general de distribuciones [20], mientras que Jacobs identifica específicamente la mónada de distribuciones discretas finitas como una mónada conmutativa [17, sección 3.1]. Esta relación proporciona la base para estudiar a continuación las mónadas de probabilidad, sin implicar que toda construcción monádica probabilística sea necesariamente conmutativa.

2.5. Mónadas de Probabilidad

Una mónada de probabilidad permite representar un cómputo como una distribución sobre sus posibles resultados. En vez de producir un único valor, cada operación puede introducir alternativas y asignar una probabilidad a cada una de ellas; la composición monádica combina posteriormente estas alternativas y propaga sus probabilidades. Para distribuciones discretas finitas, este enfoque permite enumerar de forma exacta el soporte inducido por un programa y ha sido utilizado para expresar modelos probabilísticos de manera composicional en lenguajes

funcionales [6, 31].

El ejemplo de la moneda y el dado presentado en el Código 1.1 constituye el caso más simple de esta interpretación. Sus dos elecciones uniformes generan los doce pares posibles y cada par recibe probabilidad $1/12$. Más que repetir ese ejemplo, esta sección se concentra en la representación concreta de las distribuciones y en operaciones que permiten construir modelos con elecciones uniformes, pesos no uniformes y dependencias entre variables.

La implementación utilizada en esta memoria pertenece al paquete externo `probability`, publicado en Hackage. Por lo tanto, no forma parte de los módulos estándar de `Haskell` ni del paquete `base`. Su módulo público principal es `Numeric.Probability.Distribution`, importado habitualmente con el calificador `Dist`; este módulo proporciona el tipo `T`, sus instancias de `Functor`, `Applicative` y `Monad`, y constructores de distribuciones como `certainly`, `choose`, `uniform` y `fromFreqs` [7].

La definición esencial del tipo `T` se muestra en el Código 2.16. Una distribución se almacena internamente como una lista de pares: el primer componente de cada par es un resultado posible y el segundo es su probabilidad asociada [7].

```
newtype T prob a = Cons
  { decons :: [(a, prob)] }
```

Código 2.16: Representación interna de una distribución dentro del paquete `probability`.

El parámetro `a` corresponde al tipo de los resultados y `prob` al tipo usado para sus probabilidades. En particular, `Dist.T Rational a` representa resultados de tipo `a` con pesos racionales. El uso de `Rational` permite conservar exactamente fracciones como $1/3$ o $3/10$, sin introducir los errores de redondeo propios de una representación de punto flotante. Por ejemplo, el constructor `Dist.Cons` puede almacenar los resultados `x` e `y` en una lista con los pesos $1/4$ y $3/4$, respectivamente.

Esta lista registra ramas de cómputo y no necesariamente constituye una representación canónica de la distribución. Un mismo resultado puede aparecer varias veces si fue alcanzado mediante caminos distintos. Además, el orden de los pares refleja el orden en que esas ramas fueron generadas. La semántica probabilística se obtiene al considerar, para cada resultado, la suma de los pesos de todas las apariciones que lo producen.

Las operaciones del módulo construyen estas listas de manera controlada. Una elección determinista puede escribirse como `Dist.certainly x` y produce el único par $(x, 1)$. La expresión `Dist.choose p x y` produce dos ramas: `x` con probabilidad p e `y` con probabilidad $1 - p$. Por su parte, `Dist.uniform` asigna peso $1/n$ a cada elemento de una lista de largo n , mientras que `Dist.fromFreqs` recibe pares de valores y frecuencias, y divide cada frecuencia por la suma total para obtener probabilidades. Así, las frecuencias 5, 3 y 2 se transforman respectivamente en las probabilidades $1/2$, $3/10$ y $1/5$ [7].

La instancia de `Monad` determina cómo se propagan estos pesos. Si una distribución contiene una rama (x, p) y el cómputo posterior aplicado a `x` contiene una rama (y, q) , el operador `(>>=)` incorpora el resultado `y` con peso pq . En consecuencia, la composición enumera las

combinaciones posibles y multiplica las probabilidades a lo largo de cada camino. Cuando varios caminos terminan en el mismo valor, sus pesos permanecen inicialmente en pares separados.

El Código 2.17 combina las tres formas de construcción anteriores. Primero define una distribución ponderada para el clima. Luego usa `Dist.choose` para seleccionar una actividad según el clima obtenido y, finalmente, elige uniformemente su duración. La distribución retornada omite el clima, que actúa como una variable intermedia del modelo.

```
import qualified Numeric.Probability.Distribution as Dist

data Weather = Sunny | Cloudy | Rainy
  deriving (Eq, Ord, Show)

data Activity = Outdoor | Indoor
  deriving (Eq, Ord, Show)

weekendPlan :: Dist.T Rational (Activity, Int)
weekendPlan = do
  weather <- Dist.fromFreqs
    [(Sunny, 5), (Cloudy, 3), (Rainy, 2)]
  activity <- case weather of
    Sunny  -> Dist.choose (4 / 5) Outdoor Indoor
    Cloudy  -> Dist.choose (1 / 2) Outdoor Indoor
    Rainy   -> Dist.choose (1 / 10) Outdoor Indoor
  hours <- Dist.uniform [1, 2, 3]
  return (activity, hours)

normalizedWeekendPlan = Dist.norm weekendPlan
```

Código 2.17: Modelo probabilístico con frecuencias, elecciones condicionadas y una distribución uniforme.

Antes de normalizar, este programa genera dieciocho ramas: tres posibilidades para el clima, dos para la actividad y tres para la duración. Sin embargo, como el clima no forma parte del resultado, solo existen seis pares distintos de actividad y duración. Por ejemplo, el resultado `(Outdoor, 1)` puede alcanzarse desde cualquiera de los tres climas. La suma exacta de sus ramas es

$$\frac{1}{3} \left(\frac{1}{2} \frac{4}{5} + \frac{3}{10} \frac{1}{2} + \frac{1}{5} \frac{1}{10} \right) = \frac{19}{100}.$$

Expresión 2.1: Cálculo de la probabilidad del resultado `(Outdoor, 1)`.

La función `Dist.norm` agrupa los resultados iguales y suma sus probabilidades, produ-

ciendo en este caso una lista canónica de seis resultados. En este módulo, esta operación de normalización corresponde específicamente al agrupamiento de resultados repetidos; no vuelve a dividir pesos arbitrarios para hacer que sumen uno. Esa segunda operación sí ocurre al construir una distribución mediante `Dist.fromFreqs` [7].

La distinción entre distribución y lista interna es relevante para la conmutatividad. Al intercambiar dos cálculos probabilísticos independientes, el producto de sus probabilidades se conserva, aunque el recorrido de las ramas puede producir los pares en un orden distinto. Por ello, la equivalencia considerada en esta memoria se establece sobre las distribuciones normalizadas y no sobre el orden literal expuesto por `decons`. Bajo esta interpretación, la mónada de distribuciones discretas finitas constituye el caso conmutativo descrito en la sección anterior [17, sección 3.1].

2.6. ApplicativeDo

`ApplicativeDo` es una extensión de `GHC` basada en el trabajo de Marlow, Peyton Jones, Kmett y Mokhov [25]. Su objetivo es permitir que programas escritos con *do-notation* usen combinadores aplicativos cuando las dependencias lo permiten. De esta forma, el programador conserva la sintaxis monádica habitual, mientras el compilador puede generar una estructura de ejecución más paralelizable.

La transformación que implementa `ApplicativeDo` recibe un bloque `do`, analiza las dependencias entre sus statements y produce una expresión equivalente que utiliza operaciones aplicativos donde existe independencia, sin dejar de utilizar operaciones monádicas donde un cómputo depende del resultado de otro. Para ello, el algoritmo se organiza conceptualmente como un *pipeline* de dos etapas: *rearrange* construye un plan que combina secuencias y agrupaciones aplicativos, mientras que *desugaring* traduce ese plan a combinadores explícitos [25, pp. 95–96].

Esta separación también forma parte del diseño de la implementación. La primera etapa conserva la estructura de los statements y registra cómo deben agruparse; la segunda consume esa información para producir las operaciones que implementan el plan. A continuación se profundizará en el diseño conceptual de las etapas ya mencionadas *rearrange* y *desugaring*, aplicando luego el algoritmo de `ApplicativeDo` al programa probabilístico del Código 1.2.

2.6.1. Etapa de Rearrange

Para presentar el algoritmo, considérese una expresión de *do-notation* cuyos statements de enlace tienen la forma `p_i <- e_i`. Cada statement se denota mediante s_i , donde i indica su posición dentro del bloque. El algoritmo representa entonces la expresión mediante la forma abstracta `do l e`, en la que $l = \{s_1; \dots; s_n\}$ es la secuencia de statements y e es la expresión final que produce el resultado. Esta última corresponde al argumento de un `return` o `pure` terminal y no participa en los cálculos de *rearrange*.

La Figura 2.1 presenta las funciones que componen esta etapa. La función principal,

rearrange, recibe la secuencia l y construye otra secuencia l' que puede contener la forma paralela $(l_1 \mid \dots \mid l_k)$. Esta forma no pertenece a la sintaxis escrita por el programador, sino a una sintaxis abstracta intermedia que expresa que las secuencias $l_1, \dots, l_k$ son independientes y pueden combinarse mediante operaciones aplicativas.

$$\begin{aligned} \text{rearrange } \{s_1; \dots; s_n\} &= \{s_1\}, & \text{if } n = 1, \\ &= \text{split } g_1, & \text{if } k = 1, \\ &= \{(\text{split } g_1 \mid \dots \mid \text{split } g_k)\}, & \text{otherwise} \end{aligned}$$

where

$$g_1 \dots g_k = \text{segments } \{s_1; \dots; s_n\}$$

$$\begin{aligned} \text{segments } \{s_1; \dots; s_n\} &= \{s_1; \dots; s_{i_1}\} \dots \{s_{(i_k)+1}; \dots; s_n\} \\ \text{where} \\ i_1 \dots i_k &= \{i \in 1 \dots n \\ &\mid \text{bv } \{s_1; \dots; s_i\} \cap \text{fv } \{s_{i+1}; \dots; s_n\} = \emptyset\} \end{aligned}$$

$$\begin{aligned} \text{split } \{s_1; \dots; s_n\} &= \{s_1\}, & \text{if } n = 1, \\ &= \text{splitat } i_{\text{opt}}, & \text{otherwise} \end{aligned}$$

where

$$\begin{aligned} \text{splitat } i &= \\ \text{rearrange } \{s_1; \dots; s_i\} &++ \text{rearrange } \{s_{i+1}; \dots; s_n\} \\ i_{\text{opt}} &\in 1 \dots n \text{ such that} \\ \forall j. 1 \leq j < n. &\text{cost}_s(\text{splitat } j) \geq \text{cost}_s(\text{splitat } i_{\text{opt}}) \end{aligned}$$

$$\text{cost}_s \{s_1; \dots; s_n\} = \sum \{\text{cost}_a s_i \mid 1 \leq i \leq n\}$$

$$\begin{aligned} \text{cost}_a(p \leftarrow e) &= 1 \\ \text{cost}_a(l_1 \mid \dots \mid l_n) &= \max \{\text{cost}_s l_i \mid 1 \leq i \leq n\} \end{aligned}$$

$$\begin{aligned} \text{fv } \{s_1; \dots; s_n\} &= \text{the free variables of } s_1 \dots s_n \\ \text{bv } \{s_1; \dots; s_n\} &= \text{the variables bound by } s_1 \dots s_n \end{aligned}$$

Figura 2.1: Etapa de *rearrange* propuesta por Marlow et al. [25, p. 95].

Si la entrada contiene un único statement, **rearrange** lo retorna sin modificaciones. Para una secuencia de mayor tamaño, primero invoca a **segments**, que examina las $n - 1$ separaciones posibles entre statements consecutivos. Una separación ubicada después de s_i divide la entrada en el prefijo $\{s_1; \dots; s_i\}$ y el sufijo $\{s_{i+1}; \dots; s_n\}$. El corte es válido cuando se cumple:

$$\text{bv}(\{s_1; \dots; s_i\}) \cap \text{fv}(\{s_{i+1}; \dots; s_n\}) = \emptyset,$$

Expresión 2.2: Condición de independencia entre el prefijo y el sufijo de una secuencia.

En la notación `bv` representa las variables vinculadas por una secuencia y `fv` sus variables libres. Si la intersección es vacía, ninguna variable producida por el prefijo es requerida por el sufijo y, por tanto, no existe una dependencia que atraviese esa separación. Al considerar todos los cortes válidos, **segments** divide la entrada en los grupos $g_1, \dots, g_k$ que ya pueden formar las ramas de una agrupación aplicativa.

La condición anterior expresa una relación *def-use*. Esta relación establece que el uso de una variable debe encontrarse después de su definición y dentro del alcance introducido por ella. Si una variable definida en $\{s_1; \dots; s_i\}$ aparece libre en $\{s_{i+1}; \dots; s_n\}$, ambos fragmentos no pueden tratarse como ramas independientes: el segundo necesita un resultado producido por el primero. En cambio, una intersección vacía indica que el corte no separa ninguna definición de sus usos. Como la secuencia original ya está bien formada, una variable definida en el sufijo tampoco puede ser utilizada por un statement anterior del prefijo.

Cuando **segments** obtiene más de un grupo, **rearrange** construye la forma `(split  $g_1$  |  $\dots$  | split  $g_k$ )`. Cada grupo se procesa mediante **split** para descubrir posibles agrupaciones anidadas. Si, por el contrario, toda la entrada constituye un único segmento, no existe un corte exterior que permita combinar dos partes aplicativamente y **rearrange** delega directamente la secuencia completa a **split**.

En su formulación óptima, la función **split** considera nuevamente las separaciones $i \in \{1, \dots, n-1\}$, pero con un propósito distinto. Para cada i , aplica **rearrange** recursivamente al prefijo y al sufijo, y luego compone ambos resultados de manera secuencial. Entre estas alternativas selecciona aquella que minimiza el costo definido por el modelo estructural de la Expresión 1.1. De esta manera, **segments** prioriza la independencia exterior que puede expresarse directamente mediante una agrupación aplicativa, mientras que **split** determina dónde conservar una composición monádica cuando una dependencia impide realizar esa agrupación.

Para realizar esta tarea, **GHC** implementa dos estrategias. La estrategia predeterminada evita comparar exhaustivamente todos los puntos de separación de cada segmento indivisible y selecciona uno mediante una heurística, por lo que presenta una complejidad temporal de $O(n^2)$. Aunque esta estrategia suele encontrar el plan de menor costo, no lo garantiza en todos los casos. Por otra parte, la opción de compilación `-foptimal-applicative-do` activa una estrategia basada en programación dinámica, que reutiliza los resultados calculados para las subsecuencias y garantiza la obtención de un plan de costo estructural mínimo. Esta variante tiene complejidad temporal $O(n^3)$ [13].

La selección entre ambas estrategias se realiza antes de comenzar la construcción del plan de ejecución. Las dos comparten la lógica de **segments** para identificar grupos independientes, pero difieren en la estrategia utilizada para dividir los segmentos indivisibles mediante **split**. Si n es la cantidad de statements procesados por **rearrange**, sus complejidades temporales se resumen como:

$$T_{\text{rearrange}}(n) = \begin{cases} O(n^2), & \text{variante heurística predeterminada,} \\ O(n^3), & \text{variante óptima con programación dinámica.} \end{cases}$$

Expresión 2.3: Complejidad temporal de las variantes de *rearrange*.

Finalmente, el resultado de la etapa *rearrange* es la expresión abstracta `do l' e`. Aunque *l'* puede tener una estructura de árbol con secuencias y agrupaciones aplicativas, el algoritmo no permuta statements: si se eliminan las formas paralelas y se concatenan sus ramas de izquierda a derecha, se recupera la secuencia original $\{s_1; \dots; s_n\}$ [25, p. 95].

2.6.2. Etapa de Desugaring

La segunda etapa recibe el plan *l'* construido por *rearrange* junto con la expresión final *e*. Su función principal puede representarse mediante la firma:

`desugar :: Stmts -> Expr -> Expr`

Expresión 2.4: Firma abstracta de la función `desugar`.

El segundo argumento actúa como la continuación del plan: es la expresión que debe evaluarse una vez ejecutados los efectos de la secuencia y vinculadas las variables que utiliza.

La sintaxis intermedia permite que un statement sea un enlace ordinario o una agrupación ($l_1 \mid \dots \mid l_n$). En esta última, cada l_i es a su vez una secuencia de statements. Las variables definidas dentro de una rama no quedan disponibles para las demás ramas, pero aquellas requeridas por la continuación deben ser entregadas como resultados de esa rama. Esta restricción es la que permite reemplazar la agrupación por una expresión construida con `(<$>)` y `(<*>)`.

La Figura 2.2 define el proceso mediante cinco casos recursivos. En conjunto, estos casos eliminan la sintaxis intermedia y producen una expresión formada por operaciones de `Functor`, `Applicative` y `Monad`.

$\text{desugar} :: \text{Stmts} \rightarrow \text{Expr} \rightarrow \text{Expr}$

$$\text{desugar } \{\} e = \text{pure } e \quad (0)$$

$$\begin{aligned} \text{desugar } \{p \leftarrow e\} e' & \quad (1) \\ | \quad p == e' & = e \\ | \quad \text{otherwise} & = (\lambda p \rightarrow e') \langle \$ \rangle e \end{aligned}$$

$$\text{desugar } \{p \leftarrow e; l\} e' = e \gg= \lambda p \rightarrow \text{desugar } l e' \quad (2)$$

$$\begin{aligned} \text{desugar } \{(l_1 \mid \dots \mid l_n)\} e & \quad (3) \\ = (\lambda p_1 \dots p_n \rightarrow e) \langle \$ \rangle e_1 \langle * \rangle \dots \langle * \rangle e_n \\ \text{where } (p_i, e_i) & = \text{desugar}_{\text{arg}} l_i \text{fv}(e) \end{aligned}$$

$$\begin{aligned} \text{desugar } \{(l_1 \mid \dots \mid l_n); s\} e & \quad (4) \\ = \text{join}((\lambda p_1 \dots p_n \rightarrow e') \langle \$ \rangle e_1 \langle * \rangle \dots \langle * \rangle e_n) \\ \text{where} & \\ e' = \text{desugar } s e & \\ (p_i, e_i) = \text{desugar}_{\text{arg}} l_i \text{fv}(e') & \end{aligned}$$

$$\begin{aligned} \text{desugar}_{\text{arg}} & :: \text{Stmts} \rightarrow \text{Set Var} \rightarrow (\text{Pat}, \text{Expr}) \\ \text{desugar}_{\text{arg}} \{p \leftarrow e\} vs & = (p, e) \\ \text{desugar}_{\text{arg}} l vs & = ((v_1, \dots, v_k), \text{desugar } l (v_1, \dots, v_k)) \\ \text{where} & \\ v_1 \dots v_k & = \text{bv}(l) \cap vs \end{aligned}$$

Figura 2.2: Etapa de *desugaring* propuesta por Marlow et al. [25, p. 96].

Caso (0). Si la secuencia de statements está vacía, no queda ningún efecto por ejecutar y la continuación se introduce en el contexto mediante **pure**.

Caso (1). Si solo queda un enlace $p \leftarrow e$, el resultado de e se transforma mediante $\langle \$ \rangle$ para producir la continuación. Cuando la continuación coincide exactamente con el patrón p , el algoritmo retorna e directamente y evita introducir una transformación identidad innecesaria.

Caso (2). Si un enlace $p \leftarrow e$ está seguido por más statements, estos pueden depender del valor asociado a p . Por ello se utiliza $\gg=$ y el resto de la secuencia se desazucara recursivamente dentro de la función que recibe dicho valor.

Caso (3). Si el único elemento restante es una agrupación $(l_1 \mid \dots \mid l_n)$, cada rama se convierte en un argumento independiente. Sus resultados se reciben mediante una función anónima que construye la continuación, y las expresiones de las ramas se combinan usando $\langle \$ \rangle$ y una sucesión de $\langle * \rangle$.

Caso (4). Si una agrupación aplicativa está seguida por otros statements, primero se desazucara ese resto para obtener una continuación que todavía produce un cómputo

contextualizado. La combinación aplicativa genera entonces un cómputo anidado, por lo que `join` elimina un nivel del contexto y permite continuar con el plan.

La función auxiliar `desugar_arg` determina el patrón y la expresión correspondientes a cada rama de una agrupación. Para ello, intersecta las variables definidas por la rama con las variables libres de la continuación. Así, una rama compuesta puede realizar varios enlaces internos, pero solo entrega al exterior los resultados que serán utilizados posteriormente. Luego, su propia secuencia se procesa recursivamente mediante `desugar`, lo que permite conservar dependencias monádicas dentro de una rama y combinarlas aplicativamente con las demás.

2.6.3. Aplicación del algoritmo

Para ilustrar ambas etapas se retomará el modelo probabilístico del estudiante presentado en el Código 1.2. Con el fin de concentrar la derivación en la estructura del programa, las cuatro expresiones monádicas del lado derecho de los enlaces se representarán temporalmente mediante `expr1`, `expr2`, `expr3` y `expr4`:

```
studentResults = do
  preparation <- expr1
  quiz <- expr2
  difficulty <- expr3
  finalExam <- expr4
  return (preparation, quiz, difficulty, finalExam)
```

Código 2.18: Representación abreviada del modelo probabilístico para aplicar `ApplicativeDo`.

Estas expresiones son metavariables y no definiciones independientes de Haskell: `expr2` conserva el uso libre de `preparation`, mientras que `expr4` conserva los usos de `preparation` y `difficulty`. Por su parte, `expr1` y `expr3` no utilizan variables definidas por otros statements del bloque. Denotando los cuatro statements por s_1 , s_2 , s_3 y s_4 , respectivamente, la entrada abstracta de *rearrange* es:

$$\text{do } \{s_1; s_2; s_3; s_4\} (\text{preparation}, \text{quiz}, \text{difficulty}, \text{finalExam}).$$

Expresión 2.5: Entrada abstracta de *rearrange* para el modelo probabilístico del estudiante.

La llamada inicial a `segments` examina los tres posibles puntos de corte. El corte después de s_1 no es válido porque `preparation` se utiliza en s_2 y s_4 . El corte después de s_2 tampoco lo es, pues s_4 todavía utiliza `preparation`. Finalmente, el corte después de s_3 separaría las definiciones de `preparation` y `difficulty` de sus usos en s_4 . En consecuencia, los cuatro statements forman un único segmento y se aplica `split`.

Para las separaciones $i \in \{1, 2, 3\}$, las llamadas recursivas a **rearrange** producen los siguientes planes y costos:

$$\begin{aligned} i = 1 & \implies \{s_1; (s_2 \mid \{s_3; s_4\})\} & C = 3, \\ i = 2 & \implies \{s_1; s_2; s_3; s_4\} & C = 4, \\ i = 3 & \implies \{(\{s_1; s_2\} \mid s_3); s_4\} & C = 3. \end{aligned}$$

Expresión 2.6: Planes y costos obtenidos por **rearrange** para las tres separaciones posibles.

La primera y la tercera alternativa tienen el mismo costo mínimo. Para continuar la derivación se toma la primera, que coincide con el plan producido por **GHC** para el orden original, mostrado también en la Tabla 1.2. Al sustituir nuevamente los patrones de los statements, el objeto entregado a la etapa de *desugaring* tiene la siguiente estructura intermedia:

```
do {
  preparation <- expr1;
  (quiz <- expr2 |
   { difficulty <- expr3;
     finalExam <- expr4 })
} (preparation, quiz, difficulty, finalExam)
```

Código 2.19: Plan intermedio producido por la etapa de *rearrange*.

El primer enlace permanece secuencial porque los dos componentes posteriores necesitan **preparation**. Después de obtener ese valor, la expresión asociada al quiz puede combinarse aplicativamente con la secuencia que obtiene la dificultad y, a partir de ella, el resultado del examen final. Al aplicar las reglas de *desugaring*, se obtiene primero la expresión abreviada:

```

studentResults =
  expr1 >>= \preparation ->
    (\quiz (difficulty, finalExam) ->
      (preparation, quiz, difficulty, finalExam))
  <$> expr2
  <*> (expr3 >>= \difficulty ->
    (\finalExam -> (difficulty, finalExam))
    <$> expr4)

```

Código 2.20: Resultado del desazucarado expresado mediante `expr1–expr4`.

En esta expresión, `expr2` se evalúa dentro del alcance de `preparation`. La segunda rama conserva un enlace monádico entre `expr3` y `expr4`, ya que la última requiere `difficulty`; el par producido por esa rama permite entregar ambos valores a la función ubicada a la izquierda de `(<$>)`. Este caso no requiere `join`, porque la agrupación aplicativa es el último componente del plan.

Finalmente, al reemplazar cada metavariable por la expresión correspondiente del programa original, se obtiene la forma explícita generada conceptualmente por las dos etapas de `ApplicativeDo`:

```

studentResults =
  Dist.uniform [Low, High] >>= \preparation ->
    (\quiz (difficulty, finalExam) ->
      (preparation, quiz, difficulty, finalExam))
  <$> quizGiven preparation
  <*> (Dist.uniform [Easy, Hard] >>= \difficulty ->
    (\finalExam -> (difficulty, finalExam))
    <$> examGiven preparation difficulty)

```

Código 2.21: Desazucarado aplicativo del modelo probabilístico del estudiante.

La expresión resultante combina operaciones monádicas y aplicativas según las dependencias del programa, pero al aplanar su plan conserva el orden relativo `[1, 2, 3, 4]`. La posibilidad de evaluar reordenamientos válidos bajo una hipótesis explícita de conmutatividad queda fuera del algoritmo descrito y constituye la limitación que se desarrolla en el capítulo siguiente.

Capítulo 3

Problema

En este capítulo se explicará la limitación del orden sintáctico de `ApplicativeDo`. Primero se establecerá por qué preservar el orden escrito es necesario para mantener la corrección sobre una mónada arbitraria. Luego se retomará el ejemplo probabilístico de la Sección 1.2 para mostrar que, bajo una hipótesis explícita de conmutatividad, un reordenamiento válido puede exponer un plan de menor costo. Finalmente, se formulará la oportunidad que surge de esta diferencia entre el caso general y el caso conmutativo.

3.1. Limitación del orden sintáctico de `ApplicativeDo`

La implementación estándar de `ApplicativeDo` transforma bloques `do` preservando el orden relativo de los statements escritos por el programador. Esta restricción es correcta para mónadas arbitrarias: si un bloque contiene efectos observables, como escrituras por pantalla, lectura de archivos o mutación de estado, intercambiar dos acciones puede cambiar el comportamiento del programa.

El costo de esa seguridad es que el algoritmo solo puede descubrir paralelismo cuando este aparece compatible con el orden sintáctico original. Si dos subcómputos independientes no están ubicados en posiciones convenientes, `ApplicativeDo` puede generar un plan mejor que la ejecución secuencial, pero no necesariamente el mejor plan disponible bajo una hipótesis más fuerte. En particular, bajo una hipótesis explícita de conmutatividad, ciertos statements independientes pueden cambiar de posición sin alterar la semántica. Esa posibilidad no está explotada por el algoritmo base de Marlow et al., porque su diseño debe seguir siendo válido para cualquier mónada [25].

La limitación estudiada en esta memoria aparece precisamente en esa diferencia: preservar el orden es necesario en el caso general, pero puede ser demasiado conservador cuando el programa declara explícitamente que el bloque se evalúa en una mónada conmutativa. El problema consiste entonces en determinar los reordenamientos válidos y si, bajo la hipótesis explícita de conmutatividad que permite considerarlos semánticamente equivalentes al orden original, alguno de ellos permite que `ApplicativeDo` construya un plan de menor costo.

3.2. Evidencia del problema y oportunidad

Para evidenciar la limitación anterior se retoma el modelo probabilístico del estudiante presentado en el Código 1.2. La Sección 2.6.3 aplicó el algoritmo de `ApplicativeDo` a su orden original $[1, 2, 3, 4]$ y obtuvo un plan de costo 3. En esta subsección se conserva el mismo programa y se estudia qué ocurre al intercambiar únicamente s_2 y s_3 .

Se utilizarán nuevamente las metavariables `expr1`, `expr2`, `expr3` y `expr4` introducidas en el Código 2.18. Estas mantienen las variables libres de las expresiones originales, pero permiten concentrar el análisis en la estructura del bloque. Después del intercambio, el programa abreviado queda en el orden $[1, 3, 2, 4]$:

```
studentResults = do
  preparation <- expr1
  difficulty <- expr3
  quiz <- expr2
  finalExam <- expr4
  return (preparation, quiz, difficulty, finalExam)
```

Código 3.1: Representación abreviada del modelo probabilístico con los statements `s2` y `s3` intercambiados.

El reordenamiento $[1, 3, 2, 4]$ es válido porque conserva las precedencias del programa. El statement s_1 permanece antes de s_2 y s_4 , mientras que s_3 permanece antes de s_4 , tal como muestra la Figura 1.2. Por lo tanto, el intercambio no separa ninguna definición de sus usos. Además, en el bloque no aparecen escrituras, mutación de estado ni otros efectos observables sensibles al orden: adelantar la elección de la dificultad no modifica las relaciones condicionales modeladas por `quizGiven` y `examGiven`. Bajo la hipótesis explícita de conmutatividad adoptada para la mónada de distribuciones, este reordenamiento válido se considera semánticamente equivalente al orden original.

La entrada abstracta de `rearrange` para el reordenamiento válido $[1, 3, 2, 4]$ es:

$$\text{do } \{s_1; s_3; s_2; s_4\} (\text{preparation}, \text{quiz}, \text{difficulty}, \text{finalExam}).$$

Expresión 3.1: Entrada abstracta de `rearrange` para el reordenamiento válido $[1, 3, 2, 4]$.

La llamada inicial a `segments` tampoco encuentra un corte exterior para esta secuencia. Después de s_1 , la variable `preparation` todavía es requerida por s_2 y s_4 . Después de s_3 , tanto `preparation` como `difficulty` son utilizadas en el sufijo. Finalmente, después de s_2 , ambas variables siguen siendo necesarias para s_4 . En consecuencia, los cuatro statements forman un único segmento y `rearrange` delega la secuencia a `split`.

La función `split` evalúa las separaciones $i \in \{1, 2, 3\}$. Para cada una aplica recursivamen-

te **rearrange** al prefijo y al sufijo, y obtiene los siguientes planes de acuerdo con el modelo de costos de la Expresión 1.1:

$$\begin{aligned} i = 1 & \implies \{s_1; \{s_3; (s_2 \mid s_4)\}\} & C = 3, \\ i = 2 & \implies \{(s_1 \mid s_3); (s_2 \mid s_4)\} & C = 2, \\ i = 3 & \implies \{\{s_1; (s_3 \mid s_2)\}; s_4\} & C = 3. \end{aligned}$$

Expresión 3.2: Planes y costos obtenidos por **split** para el reordenamiento válido [1, 3, 2, 4].

La separación $i = 2$ divide la secuencia en el prefijo $\{s_1; s_3\}$ y el sufijo $\{s_2; s_4\}$. Al ejecutar **rearrange** sobre el prefijo, **segments** determina que s_1 y s_3 son independientes y construye $(s_1 \mid s_3)$. Una vez disponibles **preparation** y **difficulty**, la llamada sobre el sufijo también determina que s_2 y s_4 son independientes entre sí y construye $(s_2 \mid s_4)$. Cada agrupación tiene costo 1 y ambas se componen secuencialmente, por lo que el costo total es $1 + 1 = 2$. Este es el plan mínimo seleccionado por **split**.

Al sustituir los patrones de los statements, la etapa de *rearrange* entrega a *desugaring* la siguiente estructura intermedia:

```
do {
  (preparation <- expr1 |
   difficulty <- expr3);
  (quiz <- expr2 |
   finalExam <- expr4)
} (preparation, quiz, difficulty, finalExam)
```

Código 3.2: Plan intermedio producido por *rearrange* para el reordenamiento válido [1, 3, 2, 4].

La etapa de *desugaring* procesa primero la segunda agrupación, que es el último componente del plan, y combina **expr2** y **expr4** mediante $(\<\$>)$ y $(\<*>)$. La primera agrupación está seguida por ese cómputo contextualizado, por lo que corresponde al caso que produce un contexto anidado. La operación **join** elimina ese nivel adicional y permite secuenciar ambas agrupaciones. El resultado abreviado es:


```

studentResults =
  join $
    (\preparation difficulty ->
      (\quiz finalExam ->
        (preparation, quiz, difficulty, finalExam))
      <$> expr2
      <*> expr4)
    <$> expr1
    <*> expr3

```

Código 3.3: Desazucarado del plan obtenido a partir del reordenamiento válido [1,3,2,4], expresado mediante `expr1–expr4`.

Finalmente, al reemplazar las metavariables por las expresiones del Código 1.2, se obtiene:

```

studentResults =
  join $
    (\preparation difficulty ->
      (\quiz finalExam ->
        (preparation, quiz, difficulty, finalExam))
      <$> quizGiven preparation
      <*> examGiven preparation difficulty)
    <$> Dist.uniform [Low, High]
    <*> Dist.uniform [Easy, Hard]

```

Código 3.4: Desazucarado aplicativo del modelo probabilístico obtenido a partir del reordenamiento válido [1,3,2,4].

En comparación con el último desazucarado del orden original, presentado en el Código 2.21, la nueva expresión aumenta de una a dos apariciones de (`<*>`). La primera combina la preparación con la dificultad y la segunda combina el quiz con el examen final. Este mayor uso de la estructura applicativa concuerda con el plan de costo 2: el reordenamiento válido [1,3,2,4] expone dos pares de cómputos independientes donde el orden original solo permitía una agrupación applicativa exterior. Por consiguiente, existe una oportunidad de mejora que el algoritmo estándar no puede aprovechar mientras conserve exclusivamente el orden escrito por el programador.

3.3. Formulación del problema

El algoritmo de `ApplicativeDo` transforma correctamente programas sobre una mónada arbitraria. Como sus efectos pueden ser observables y sensibles al orden, preserva la secuencia relativa de los statements escritos por el programador. Esta decisión permite introducir combinadores applicativos sin cambiar el comportamiento que tendría el desazucarado monádico original.

La situación es menos restrictiva cuando la mónada es conmutativa. Si dos statements son independientes, su intercambio puede preservar tanto las dependencias del programa como su significado. La evidencia de la subsección anterior muestra que esta libertad semántica no es solo una propiedad abstracta: un reordenamiento válido, considerado semánticamente equivalente al orden original bajo una hipótesis explícita de conmutatividad, puede permitir que el mismo algoritmo de `ApplicativeDo` produzca un plan de menor costo que el obtenido sobre el orden original.

Sin embargo, la implementación actual de `ApplicativeDo` en `GHC` no incorpora un tratamiento especializado para este caso: aun cuando la mónada pueda tratarse bajo una hipótesis explícita de conmutatividad, no estudia reordenamientos válidos de los statements antes de construir el plan de ejecución. El problema abordado por esta memoria consiste, por tanto, en determinar cómo aprovechar esa hipótesis para ampliar el espacio de planes evaluables mediante reordenamientos válidos, sin alterar el comportamiento general requerido para mónadas arbitrarias.

Capítulo 4

Diseño de Solución

A partir del problema formulado en la Sección 3.3, se observa que, aunque `ApplicativeDo` puede mejorar el plan de ejecución completamente secuencial, la preservación del orden relativo de los statements limita el espacio de planes que puede explorar. Bajo una hipótesis explícita de conmutatividad, aplicar `ApplicativeDo` sobre un reordenamiento válido de statements puede exponer un plan de ejecución de menor costo que el obtenido a partir del orden original [25, pp. 94–95].

La solución propuesta por la memoria consiste en extender el espacio de búsqueda de `ApplicativeDo`: para un bloque `do` declarado como conmutativo, se generan los reordenamientos válidos, se aplica el algoritmo existente sobre cada uno y se selecciona el plan de ejecución de menor costo. De este modo, la extensión aprovecha la libertad adicional del bloque `do` sin reemplazar `ApplicativeDo` ni modificar el tratamiento de los bloques ordinarios.

En este capítulo se desarrolla el diseño de dicha solución. Primero, se presenta el flujo general de la extensión, desde la declaración del bloque como conmutativo hasta la selección del plan de ejecución de menor costo. A continuación, se introduce el *grafo de precedencia RAW* como representación de las restricciones entre los statements de un bloque `do`, junto con el modelo de dependencias que justifica por qué basta considerar dependencias locales de tipo *Read-after-Write*. Finalmente, se caracterizan los reordenamientos válidos como ordenamientos topológicos del grafo y se describe el recorrido exhaustivo utilizado para generarlos.

4.1. Flujo general de la extensión

El diseño propuesto amplía el flujo habitual de `ApplicativeDo` al incorporar la exploración de órdenes alternativos antes de construir el plan de ejecución. Sobre un bloque `do` habilitado, la solución representa las dependencias locales mediante un grafo de precedencia RAW. Sus aristas determinan qué relaciones deben conservarse y, por tanto, caracterizan como reordenamientos válidos a las secuencias completas que respetan todas esas precedencias. Dentro del alcance experimental de esta memoria, la hipótesis explícita de conmutatividad permite considerar dichos reordenamientos semánticamente equivalentes al orden original; la

ausencia de una dependencia local, por sí sola, no sería suficiente para autorizar el intercambio sobre una mónada arbitraria [25, p. 94].

Cada reordenamiento válido se entrega a la etapa de *rearrange* de **ApplicativeDo**. Como se explicó en la Sección 2.6.1, esta etapa conserva el orden de statements recibido y construye un árbol que combina secuencias monádicas y agrupaciones aplicativas. La extensión compara los árboles obtenidos para todos los reordenamientos mediante el modelo de costo de la Expresión 1.1 y selecciona el primer candidato de menor costo. El árbol seleccionado continúa luego hacia la etapa de *desugaring*. Por consiguiente, la contribución amplía el espacio de búsqueda, mientras que **ApplicativeDo** sigue siendo responsable de construir el plan asociado a cada reordenamiento. La Figura 4.1 resume este proceso.

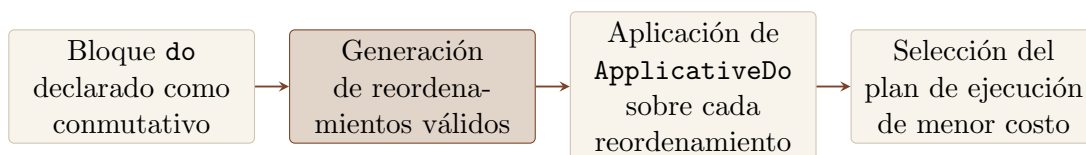


Figura 4.1: Flujo general de la extensión propuesta.

El diseño separa así dos responsabilidades. El programador afirma que el orden de efectos independientes puede intercambiarse en el bloque **do** marcado. El compilador restringe la exploración a reordenamientos que conservan las relaciones locales de definición y uso, evalúa los planes de ejecución correspondientes y selecciona el menos costoso, sin modificar la compilación de los bloques que no son declarados como conmutativos.

En este flujo, el núcleo algorítmico de la extensión corresponde a la generación de reordenamientos válidos, cuyo diseño se divide en dos fases. La primera construye el grafo de precedencia RAW que representa las restricciones entre los statements del bloque; la segunda recorre exhaustivamente dicho grafo para generar los reordenamientos que preservan esas restricciones. Ambas fases se desarrollan, respectivamente, en las Secciones 4.2 y 4.3.

4.2. Grafo de precedencia RAW

En la primera fase del proceso de generación de reordenamientos válidos se requiere una representación capaz de expresar las precedencias que deben conservarse entre los statements de un bloque **do** habilitado. Siguiendo el uso de grafos de dependencias para formular e implementar optimizaciones de compiladores [21], la solución utiliza un grafo de precedencia cuyas aristas representan exclusivamente dependencias locales de tipo *Read-after-Write*; este grafo especializado se denomina *grafo de precedencia RAW*. La Figura 1.1 presentó previamente esta representación para el programa del Código 1.2: cada vértice corresponde a un statement y cada arista conecta un statement productor con otro que consume uno de sus valores. Sin embargo, antes de definir formalmente el grafo, es necesario justificar por qué basta considerar dependencias RAW y por qué las dependencias *Write-after-Read* y *Write-after-Write* no originan precedencias adicionales entre statements renombrados. A continuación, se presenta el modelo de dependencias que fundamenta esta decisión.

4.2.1. Modelo de dependencias

El punto de partida es un modelo general de dependencias entre lecturas y escrituras. Considérense dos statements s_i y s_j tales que $i < j$. Cada statement s_k posee un conjunto R_k de variables leídas y un conjunto W_k de variables escritas. A partir de estos conjuntos se distinguen tres clases de dependencia: *Read-after-Write* (RAW), *Write-after-Read* (WAR) y *Write-after-Write* (WAW):

$$\begin{aligned}\text{RAW}(i, j) &\iff W_i \cap R_j \neq \emptyset, \\ \text{WAR}(i, j) &\iff R_i \cap W_j \neq \emptyset, \\ \text{WAW}(i, j) &\iff W_i \cap W_j \neq \emptyset.\end{aligned}$$

Expresión 4.1: Dependencias de escritura y lectura de variables entre dos statements s_i y s_j .

Una dependencia RAW establece una relación productor–consumidor: s_j lee una variable cuyo valor fue escrito por s_i . Una dependencia WAR aparece cuando s_i debe leer una variable antes de que s_j la sobrescriba; invertir ambos statements haría que la lectura observara el nuevo valor. Finalmente, una dependencia WAW relaciona dos escrituras sobre una misma variable y obliga a conservar cuál de ellas determina su valor posterior. Estas dos últimas relaciones presuponen que distintas escrituras pueden actuar sobre una misma identidad mutable, como ocurre en un modelo imperativo de asignaciones. Alterar cualquiera de estas precedencias puede modificar los valores observados o el estado final del programa [2, 5].

La *do-notation* de **Haskell** presenta un comportamiento diferente. En un statement de la forma $p \leftarrow e$, el patrón p introduce una nueva vinculación léxica, denominada *binder*; no actualiza destructivamente una variable definida con anterioridad. Si un statement posterior reutiliza el mismo identificador textual, introduce otro binder que oculta al primero en su nuevo alcance. Este comportamiento concuerda con la pureza descrita en la Sección 2.1 y con las reglas de alcance de la *do-notation* [22, sección 3.14].

El Renamer de **GHC** hace explícita esta diferencia al resolver los identificadores del programa. Cada binder recibe un **Name** único y cada uso queda asociado con la identidad que se encuentra en alcance en ese punto. Por ello, dos definiciones escritas como x en el programa fuente no representan dos escrituras internas sobre el mismo nombre [1, 11].

El Código 4.1 ilustra este comportamiento con un bloque **do** de tipo **Maybe Int**. En el programa fuente, y lee la primera vinculación de x antes de que un statement posterior vuelva a utilizar ese identificador. La columna derecha presenta esquemáticamente las identidades diferenciadas durante el renombrado del compilador:

<pre>do x <- Just 2 y <- Just ((2*) x) x <- Just 3 return (x + y)</pre>	$\Rightarrow$	<pre>do x_a1T2 <- Just 2 y_a1T3 <- Just ((2*) x_a1T2) x_a1T4 <- Just 3 return (x_a1T4 + y_a1T3)</pre>
--	---------------	--

Código 4.1: Representación esquemática de los binders antes y después del renombrado que ejecuta GHC a todo programa.

La primera y la tercera definición escriben, respectivamente, los nombres `x_a1T2` y `x_a1T4`. Como son identidades diferentes, sus conjuntos de escritura no se intersectan y, por tanto, no existe una dependencia WAW. De manera análoga, el segundo statement lee `x_a1T2`, mientras que el tercero escribe `x_a1T4`; la intersección entre esa lectura y la escritura posterior también es vacía, por lo que no existe una dependencia WAR. La relación local efectiva entre los statements reordenables es RAW: la definición de `x_a1T2` debe permanecer antes del statement que define `y_a1T3`. En consecuencia, el grafo de precedencia utilizado por la solución se construye exclusivamente a partir de dependencias RAW entre nombres renombrados.

Sea $S = (s_1, \dots, s_n)$ la secuencia de statements que participan en el reordenamiento. El statement terminal, representado internamente como `LastStmt`, no pertenece a S porque produce el resultado del bloque y permanece fijo [25, p. 95]. La anotación de variables libres que entrega el Renamer para el cuerpo de cada statement contiene los `Names` utilizados que no son introducidos por ese mismo cuerpo [11]. Esto incluye tanto referencias a binders definidos por statements anteriores como nombres globales, entre ellos funciones, operadores y constructores de datos. Por ejemplo, en el segundo statement del Código 4.1 aparecen como nombres libres `x_a1T2`, el constructor `Just` y el operador `(*)`, pero solo el primero corresponde a un valor producido dentro del bloque `do`. Para distinguir las lecturas locales de las referencias externas se definen los siguientes conjuntos para cada statement s_i :

$$\begin{aligned}
\text{WRITE}(s_i) &= \text{binders}(s_i), \\
\text{ALL_WRITES} &= \bigcup_{k=1}^n \text{WRITE}(s_k), \\
\text{READ}_{\text{local}}(s_i) &= \text{FV}(s_i) \cap \text{ALL_WRITES}.
\end{aligned}$$

Expresión 4.2: Escrituras y lecturas locales de un statement renombrado.

El conjunto $\text{WRITE}(s_i)$ contiene los `Names` de los binders introducidos por s_i , mientras que ALL_WRITES reúne todos los nombres producidos por los statements reordenables del bloque `do`. Por su parte, $\text{FV}(s_i)$ conserva el conjunto completo de nombres libres informado por el Renamer de GHC. Al intersectarlo con ALL_WRITES , $\text{READ}_{\text{local}}(s_i)$ retiene únicamente los nombres cuyo valor se origina por un binder de un statement anterior. En el ejemplo del Código 4.1, `x_a1T2` permanece como lectura local del segundo statement, mientras que `Just` y `(*)` se descartan de $\text{READ}_{\text{local}}(s_2)$ porque, aunque pertenecen a $\text{FV}(s_2)$, no forman

parte de ALL_WRITES.

Con esta información, la restricción relevante entre dos statements s_i y s_j queda dada por

$$i \longrightarrow j \iff i < j \wedge \text{WRITE}(s_i) \cap \text{READ}_{\text{local}}(s_j) \neq \emptyset.$$

Expresión 4.3: Dependencia RAW local utilizada por la extensión.

La condición $i < j$ orienta la dependencia según el orden del programa original. La intersección no vacía indica que existe al menos un **Name** definido por s_i y utilizado localmente por s_j ; dicho nombre actúa como testigo de una relación productor–consumidor que debe preservarse [16]. Debido a que la comparación se realiza sobre nombres renombrados, la arista no puede surgir solo por la reutilización de un mismo identificador textual.

El grafo de precedencia RAW de un bloque **do** se define entonces como:

$$G_{\text{RAW}} = (V, E_{\text{RAW}}), \quad V = \{1, \dots, n\}, \quad E_{\text{RAW}} = \{(i, j) \in V \times V \mid i \longrightarrow j\}.$$

Expresión 4.4: Grafo de precedencia RAW inducido por las dependencias locales de un bloque **do**.

Cada vértice representa un statement reordenable y cada arista registra una dependencia RAW local que, al mismo tiempo, impone una precedencia productor–consumidor. Como toda arista satisface $i < j$, los índices aumentan estrictamente a lo largo de cualquier camino y G_{RAW} es acíclico. El conjunto E_{RAW} contiene todas las dependencias detectadas por el criterio anterior y no corresponde necesariamente a una reducción transitiva; la relación de alcanzabilidad del grafo induce el orden parcial que debe respetar todo reordenamiento válido [30].

El grafo de precedencia RAW expresa así las restricciones estructurales que determinan la validez de un reordenamiento; por sí solo no garantiza equivalencia semántica, que en esta memoria depende de la hipótesis explícita de conmutatividad [20, secciones 5 y 9]. A partir del orden parcial inducido por este grafo, la segunda fase determina los reordenamientos de statements que preservan todas las precedencias.

4.3. Generación de reordenamientos válidos

Una vez construido el grafo de precedencia RAW G_{RAW} asociado a un bloque **do**, generar los reordenamientos válidos equivale a obtener los ordenamientos topológicos del grafo. Un ordenamiento topológico es una forma de disponer todos sus vértices en una secuencia que respeta la dirección de las aristas [18]. En este caso, si existe una arista $i \rightarrow j$, el statement s_i debe aparecer antes que s_j ; si el grafo no impone una precedencia entre dos statements,

estos pueden aparecer en cualquier orden relativo. Esta caracterización es central para el diseño de la solución: transforma la preservación de las dependencias locales en una propiedad estructural verificable sobre el grafo. En otras palabras, para determinar si un reordenamiento es válido basta comprobar que el origen de cada arista aparezca antes que su destino. Como toda arista satisface $i < j$, el orden original $(s_1, \dots, s_n)$ también es topológico y permanece entre los candidatos efectivamente evaluados.

El procedimiento construye estos ordenamientos una posición a la vez. En cada paso puede seleccionar cualquier statement que todavía no haya sido agregado y cuyos predecesores ya aparezcan en la secuencia. Si existen varios statements disponibles, el procedimiento explora cada elección como una alternativa distinta. Este proceso se repite hasta incorporar todos los statements y, al considerar todas las elecciones posibles, se obtienen todos los ordenamientos topológicos del grafo y por ende todos los reordenamientos válidos del bloque `do` [19].

Aplicado al programa del Código 1.2, este procedimiento debe preservar las tres aristas del grafo de precedencia RAW presentado en la Figura 1.1. Los cinco ordenamientos topológicos que satisfacen estas restricciones se presentan en la Tabla 4.1.

Índice	Ordenamiento topológico
0	[1,2,3,4]
1	[1,3,2,4]
2	[1,3,4,2]
3	[3,1,2,4]
4	[3,1,4,2]

Tabla 4.1: Ordenamientos topológicos del grafo de precedencia RAW de la Figura 1.1.

El primer ordenamiento conserva la secuencia original; los demás intercalan statements independientes sin invertir ninguna de las tres aristas del grafo.

Así, el grafo de precedencia RAW determina las restricciones de un bloque `do` y su recorrido exhaustivo materializa los reordenamientos válidos que se entregan a la etapa de *rearrange* de `ApplicativeDo`. El Capítulo 5 desarrolla la implementación de ambas fases dentro del Renamer de GHC.

Capítulo 5

Implementación de la Solución

Este capítulo describe cómo se materializó en **GHC** el diseño de la solución presentado en el capítulo anterior. Primero se introduce el flujo de compilación relevante de **GHC** y se identifican los puntos intervenidos; luego se presenta el mecanismo declarativo que permite reconocer un bloque conmutativo y se detalla su integración en el Renamer. A continuación se desarrollan la construcción del grafo de precedencia RAW y la generación recursiva de reordenamientos válidos. Finalmente, se explica cómo cada reordenamiento es procesado por **ApplicativeDo**, evaluado mediante el modelo de costo y reducido al plan que continúa en el flujo normal del compilador.

5.1. Flujo de compilación de GHC

Antes de ubicar la extensión dentro de **GHC**, es necesario presentar las etapas del compilador que atraviesa una expresión de **Haskell**. La descripción se concentra en el *frontend*, desde la lectura del programa hasta la producción de **Core**, porque en ese recorrido se encuentran la representación y la información utilizadas por la solución.

Al comenzar la compilación, las opciones de la línea de comandos y los pragmas **LANGUAGE** del módulo determinan las extensiones habilitadas. Esta configuración forma parte de las opciones dinámicas del compilador y es consultada por las etapas que requieren cada característica. Por ejemplo, el Parser comprueba que **QualifiedDo** permita la sintaxis de un bloque **do** calificado, mientras que el Renamer consulta **ApplicativeDo** antes de ejecutar su postprocesamiento. Por tanto, la comprobación de las extensiones no constituye una transformación separada del programa, sino una condición que guía el comportamiento de las fases posteriores.

La mayoría de los compiladores representan internamente un programa mediante un *árbol de sintaxis abstracta*, o AST por su nombre en inglés. Esta estructura organiza jerárquicamente construcciones como módulos, declaraciones, expresiones y statements. Sus nodos conservan la estructura necesaria para analizar y transformar el programa, sin depender directamente de la secuencia de caracteres del archivo fuente.

En **GHC**, el AST de **Haskell** está parametrizado por la etapa de compilación. Por ejemplo, una expresión se representa mediante **HsExpr p**, donde el parámetro **p** determina la información disponible en sus nodos. Los alias **GhcPs**, **GhcRn** y **GhcTc** identifican, respectivamente, las representaciones producidas por el Parser, el Renamer y el Typechecker. No se trata de tres árboles sin relación, sino de una misma familia de sintaxis cuyos campos e invariantes evolucionan a medida que el compilador incorpora nueva información. La Expresión 5.1 resume este recorrido para una expresión.

$$\begin{array}{c} \text{código fuente} \xrightarrow{\text{Parser}} \text{HsExpr GhcPs} \xrightarrow{\text{Renamer}} \text{HsExpr GhcRn} \\ \xrightarrow{\text{Typechecker}} \text{HsExpr GhcTc} \xrightarrow{\text{Desugarer}} \text{Core.} \end{array}$$

Expresión 5.1: Fases relevantes del AST de una expresión en **GHC**.

El Parser reconoce la sintaxis del programa y construye la primera representación estructurada. En particular, un bloque **do** se incorpora como una expresión **HsDo** dentro del AST, el cual contiene el alias **GhcPs**. En esta etapa los identificadores aún corresponden a los nombres escritos en el programa y no se han resuelto sus definiciones ni sus alcances.

El Renamer transforma esa representación en un nodo del AST con alias **GhcRn**. Para ello resuelve cada identificador, determina su alcance y asigna identidades únicas de tipo **Name** a los binders. También calcula las variables libres de cada statement agrupandolas en una estructura adecuada, y ejecuta el postprocesamiento de **ApplicativeDo** sobre los bloques habilitados. Esta combinación de estructura sintáctica e información de nombres es la que permite analizar las dependencias locales sin confundir identificadores textualmente iguales que corresponden a vinculaciones distintas.

El Typechecker verifica los tipos de las expresiones y las restricciones exigidas por las clases utilizadas, y registra esa información en el mismo nodo del AST pero con alias **GhcTc**. En la solución propuesta, esta etapa comprueba indirectamente la restricción **CommutativeMonad** a través de los tipos de las operaciones calificadas empleadas por el bloque. Finalmente, el Desugarer traduce la expresión tipada a **Core**, donde las agrupaciones introducidas por **ApplicativeDo** ya se expresan mediante las operaciones aplicativas y monádicas correspondientes.

La implementación interviene en dos zonas de este recorrido. Por una parte, en la infraestructura de opciones dinámicas de **GHC** se incorpora la instrumentación experimental para la selección de reordenamientos válidos forzada, la cual se explicará en el capítulo de Validación. Por otra parte, la modificación funcional se concentra en el Renamer de **GHC**, donde se detecta el marcador conmutativo asociado a la declaración explícita hecha por el programador, se generan los reordenamientos válidos y se selecciona el plan de ejecución que permanecerá en el nodo del AST renombrado.

Además, se agregaron dos nuevos módulos de **Haskell** asociados a las mónadas conmutativas llamados ambos **CommutativeDo**, estos debieron agregarse en los archivos asociados a **ghc-internal** y en **base**, junto con las entradas necesarias para exponerlos desde ambas bi-

blotecas. El Parser, el Typechecker y el Desugarer no requirieron modificaciones: la solución reutiliza el soporte que estas etapas ya proporcionaban para `QualifiedDo` y `ApplicativeDo`. La sección siguiente desarrolla la interfaz declarativa que conecta dichos módulos con la modificación del Renamer.

5.2. Activación mediante `CommutativeDo`

La implementación de la solución requiere que la hipótesis de conmutatividad sea declarada de manera explícita y local a cada bloque `do`. Para proporcionar esta interfaz se incorporaron los módulos específicos `GHC.Internal.Control.Monad.CommutativeDo`, dentro de `ghc-internal`, y `Control.Monad.CommutativeDo`, dentro de `base`. El primero contiene el soporte interno, mientras que el segundo reexporta la interfaz destinada a los programas de usuario.

Dentro de estos módulos se define la clase `CommutativeMonad`, cuya única superclase es `Monad`. Una mónada utilizada mediante esta notación debe contar con una instancia de la clase. La instancia constituye una afirmación del programador acerca de la posibilidad de intercambiar efectos independientes; `GHC` no intenta demostrar la propiedad de conmutatividad. Las operaciones exportadas por el módulo incorporan la restricción `CommutativeMonad`, por lo que su cumplimiento se comprueba posteriormente durante el typechecking.

Para hacer uso de la solución dentro de un programa de `Haskell`, se utilizó la extensión `QualifiedDo`, esta permite anteponer el nombre de un módulo a un bloque `do`, por ejemplo `M.do` [14]. Este nombre actúa como calificador e indica al compilador que debe interpretar los statements utilizando las operaciones exportadas por el módulo `M`, en lugar de las operaciones monádicas habituales sin calificar. De esta manera, la interfaz puede seleccionarse localmente para un bloque, mientras que los demás bloques `do` del programa conservan su comportamiento normal.

En esta implementación, el módulo público se importa calificadamente, habitualmente como `CD`. Un bloque declarado como conmutativo se escribe comenzando con `CD.do` y construyendo su resultado terminal mediante `CD.return`. El calificador hace que los statements utilicen las operaciones exportadas por `CommutativeDo`, cuyos tipos incorporan la restricción `CommutativeMonad`. Además de las operaciones monádicas requeridas por `QualifiedDo`, el módulo exporta las operaciones aplicativas que puede necesitar la transformación posterior de `ApplicativeDo`.

Para conectar esta interfaz con el compilador, ambos módulos exportan el marcador `__commutative_do__`. Durante el renombrado se consulta la presencia de este nombre en el módulo que califica `CD.do`. Esta comprobación es independiente de la resolución de la instancia: el marcador habilita la ruta de reordenamiento en el Renamer, mientras que la clase restringe las operaciones y se verifica en una etapa posterior. La activación requiere, además, que `ApplicativeDo` y `QualifiedDo` estén habilitados.

De esta forma, para aplicar la extensión experimental a un bloque `do` conmutativo como el presentado en el Código 2.14, se deben habilitar ambas extensiones, importar la interfaz

calificada y declarar la instancia de `CommutativeDo` correspondiente. El Código 5.1 muestra la declaración resultante.

```
{-# LANGUAGE ApplicativeDo #-}
{-# LANGUAGE QualifiedDo #-}

import qualified Control.Monad.CommutativeDo as CD

instance CD.CommutativeMonad Maybe

sumaConmutativa :: Maybe Int
sumaConmutativa = CD.do
  x <- Just 3
  y <- Just 2
  CD.return ((+) x y)
```

Código 5.1: Uso de la extensión experimental `CommutativeDo` sobre una suma monádica con `Maybe Int`.

Los pragmas habilitan los dos mecanismos requeridos por la solución, mientras que la importación proporciona el calificador `CD`. La instancia declara la hipótesis de conmutatividad para `Maybe` y el uso de `CD.do` identifica localmente el bloque que seguirá la ruta experimental. La declaración conserva los cómputos del Código 2.14 y produce el mismo resultado, `Just 5`.

Con estas condiciones establecidas, la sección siguiente detalla la bifurcación incorporada al flujo de renombrado.

5.3. Integración en el Renamer

La integración de la solución comienza en el renombramiento de toda expresión de `Haskell`, lógica implementada por la función `rnExpr`, específicamente cuando debe procesar un nodo del AST con la expresión `HsDo`. Esta debe invocar a `rnStmtsWithFreeVars` para renombrar todos los statements y asociar a cada cuerpo el conjunto de variables libres calculado durante el recorrido. A continuación, la función `postProcessStmtsForApplicativeDo` decide si debe aplicar la transformación de `ApplicativeDo` o devolver los statements sin dicho postprocesamiento.

El postprocesamiento de un bloque `do` se ejecuta cuando `ApplicativeDo` está habilitada, ingresando a la función `rearrangeForApplicativeDo`, continuando con el proceso de renombrado.

Dentro de esa función, `isCommutativeQualifiedDo` consulta si el contexto es un bloque `do` calificado y si el módulo calificador permite resolver el marcador `__commutative.do__`. Cuando el marcador no se encuentra, el orden original constituye el único candidato y `ApplicativeDo` aplica su algoritmo habitual. Cuando el marcador se encuentra, la exten-

sión genera todos los reordenamientos válidos de statements, aplica el algoritmo de *rearrange* sobre cada uno y selecciona el plan de ejecución de menor costo. La Figura 5.1 presenta las rutas de compilación del Renamer con la extensión integrada.

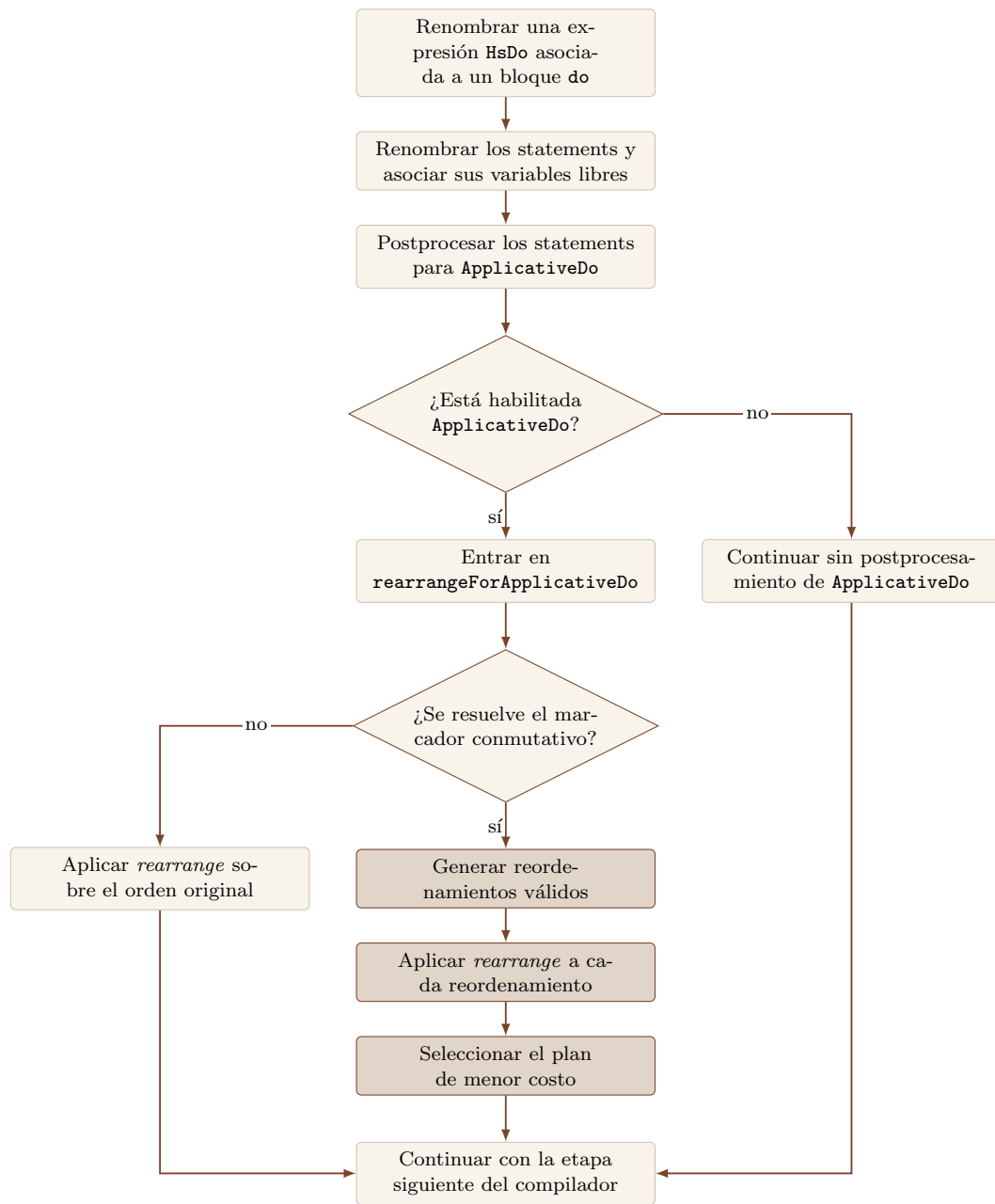


Figura 5.1: Flujo de renombrado y condiciones de activación de la ruta de reordenamiento de statements.

Dentro de ese alcance, la Figura 5.1 muestra que `ApplicativeDo` es una condición previa para ingresar en el postprocesamiento. La función `rearrangeForApplicativeDo` solo se invoca cuando esa condición se cumple; por ello, la presencia del marcador no activa por sí sola la generación de reordenamientos. Del mismo modo, `QualifiedDo` ya debió permitir la sintaxis calificada durante el parseo. Solo la combinación de un bloque calificado aceptado, `ApplicativeDo` habilitada y el marcador resoluble alcanza la ruta resaltada.

Ubicar la modificación en este punto evita realizar una reescritura externa o analizar una representación posterior que ya haya perdido la estructura del bloque `do`. También permite utilizar las identidades únicas y las variables libres producidas durante el renombrado tanto para construir las dependencias como para generar los reordenamientos válidos que recibirá `ApplicativeDo`. Las tres secciones siguientes desarrollan, en ese orden, la formación del grafo de precedencia RAW, la generación de reordenamientos válidos y la selección del plan de ejecución de menor costo.

5.4. Construcción del Grafo de Precedencia RAW

La ruta resaltada en la Figura 5.1 comienza aplicando el modelo de dependencias de la Sección 4.2.1 sobre los statements renombrados. Antes de formar el grafo, la función `rearrangeForApplicativeDo` separa el `LastStmt`, que produce el resultado del bloque `do` y permanece fijo. Considerando los demás statements $s_1, \dots, s_n$, cada uno se asocia con su posición estable, las variables libres de su cuerpo, sus binders y las lecturas locales obtenidas al restringir esas variables libres a nombres definidos dentro del mismo bloque `do`.

En vez de introducir una estructura de datos nueva, el grafo de precedencia RAW se construye con la biblioteca `GHC.Data.Graph.Directed`. Cada vértice almacena la información de un statement y utiliza su posición original como clave. Para cada par $i < j$, se agrega una arista $i \rightarrow j$ cuando los nombres escritos por s_i intersectan las lecturas locales de s_j . El Código 5.2 muestra la construcción de este grafo utilizando pseudocódigo.

```
buildRawPrecedenceGraph(statements):
  n := length(statements)
  allWrites := union(write(s) for s in statements)
  graph := graph with vertices {1, ..., n}

  for each s_i in statements:
    writesLocal[i] := write(s_i)
    readsLocal[i] := intersection(freeVars(s_i), allWrites)

  for i := 1 to n:
    for j := i + 1 to n:
      if intersection(writesLocal[i], readsLocal[j]) != empty:
        add edge i -> j to graph

  return graph
```

Código 5.2: Pseudocódigo para construir el grafo de precedencia RAW.

La condición $i < j$ conserva la precedencia del programa original y garantiza la aciclicidad del grafo, porque los índices aumentan estrictamente a lo largo de cada arista. La construcción de este realiza

$$\sum_{i=1}^{n-1} (n-i) = \frac{n(n-1)}{2}$$

comparaciones entre pares. Por tanto, la cantidad de pruebas de dependencia es $O(n^2)$, sin considerar el costo interno de construir e intersectar los conjuntos de nombres. En el ejemplo probabilístico del Código 1.2, la construcción del grafo de precedencia RAW produce las aristas $1 \rightarrow 2$, $1 \rightarrow 4$ y $3 \rightarrow 4$, presentadas en la Figura 1.1.

5.5. Generación de Reordenamientos Válidos

La función `generateAllSemanticTopSorts` recibe el grafo de precedencia RAW y materializa los reordenamientos válidos como sus ordenamientos topológicos. Para ello calcula el grado de entrada de cada vértice mediante el grafo original y su transposición. Un vértice cuyo grado de entrada es cero no posee predecesores, por lo que puede ser declarado para ser el siguiente statement en el reordenamiento calculado. Luego de ser declarado, todas las aristas salientes de este mismo son eliminadas para así garantizar que los statements originalmente dependientes de este puedan ser declarados y así continuar con la generación del ordenamiento topológico.

El algoritmo de Kahn clásico selecciona uno a uno los vértices del grafo de precedencia y produce un único ordenamiento topológico [18]. La implementación de la solución reemplaza esa elección por una ramificación sobre todos los vértices disponibles. Cada rama emite un vértice, decrementa el grado de entrada de sus sucesores y repite el procedimiento sobre el estado resultante. Cuando la secuencia acumulada de statements contiene todos los vértices del grafo, se obtiene un orden topológico y por ende un reordenamiento válido. El Código 5.3 presenta el procedimiento abstracto.

```

generateAllSemanticTopSorts(graph):
    indegree := computeIndegrees(graph)
    extendTopSorts(graph, indegree, empty, [])

extendTopSorts(graph, indegree, emitted, order):
    if length(order) == numberOfVertices(graph):
        emit order
        return

    choices := every non-emitted vertex v with indegree[v] == 0

    for each v in choices:
        nextIndegree := copy(indegree)
        for each successor w of v:
            nextIndegree[w] := nextIndegree[w] - 1

        extendTopSorts(
            graph,
            nextIndegree,
            union(emitted, {v}),
            order ++ [v])

```

Código 5.3: Pseudocódigo del algoritmo que genera todos los ordenamientos topológicos del grafo de precedencia RAW de un bloque `do`.

Cada llamada recursiva agrega únicamente un vértice cuyos predecesores ya fueron emitidos, por lo que todo resultado completo conserva las aristas del grafo. A la inversa, la primera posición de cualquier ordenamiento topológico debe tener grado de entrada cero; como el algoritmo ramifica sobre todas esas posibilidades y repite el criterio sobre el grafo restante, también alcanza cada reordenamiento válido.

Los índices se recorren en orden creciente para producir resultados estables. En particular, la primera ramificación completa del algoritmo reproduce el orden original, que queda asociado al índice cero. Si P es la cantidad de ordenamientos topológicos, la función materializa P secuencias de longitud n . En el caso extremo de un grafo sin aristas, todos los statements pueden ocupar cualquier posición y se generan $n!$ reordenamientos válidos.

La Figura 5.2 muestra la ejecución recursiva del algoritmo sobre el bloque `do` del ejemplo probabilístico con Código 1.2. Cada estado contiene el prefijo emitido π y el subgrafo $G[R]$ inducido por los vértices restantes. Una arista representa la elección del statement emitido y las hojas corresponden a los reordenamientos completos.

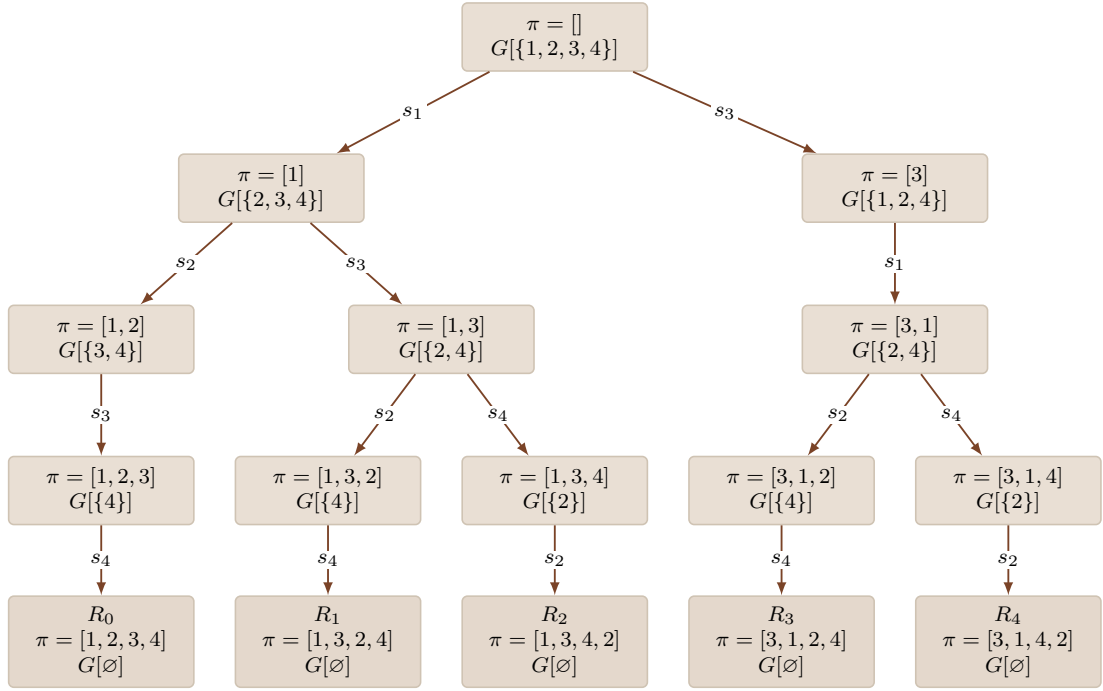


Figura 5.2: Árbol de llamadas para generar todos los reordenamientos válidos del programa probabilístico con Código 1.2. La raíz representa la invocación inicial y los otros 15 estados corresponden a llamadas recursivas.

Las cinco hojas de la figura corresponden, en el mismo orden, a los ordenamientos topológicos presentados en la Tabla 4.1. La hoja R_0 conserva la secuencia original de statements; las demás intercalan statements independientes sin invertir ninguna de las tres aristas del grafo de precedencia RAW de la Figura 1.1.

5.6. Generación y Selección de Planes

Después de la generación de todos los reordenamientos válidos de un bloque `do`, como muestra la Figura 5.1 el siguiente paso de la solución consiste en aplicar el algoritmo de *rearrange* sobre cada uno de ellos. Por ende, cada ordenamiento topológico del grafo de precedencia RAW se reconstruye como una secuencia de statements renombrados con sus anotaciones de variables libres y se entrega a la lógica existente de **ApplicativeDo**.

Todo plan de ejecución resultante se representa programáticamente mediante el tipo **StmtTree**. Sus tres constructores, mostrados en el Código 5.4, expresan un statement elemental, una composición secuencial mediante *bind* o una agrupación aplicativa.

```

data StmtTree a
  = StmtTreeOne a
  | StmtTreeBind (StmtTree a) (StmtTree a)
  | StmtTreeApplicative [StmtTree a]

```

Código 5.4: Representación de los planes de ejecución generados por `ApplicativeDo`.

La extensión obtiene un árbol por cada reordenamiento válido. Si P es la cantidad de secuencias generadas y cada una contiene n statements, la etapa de *rearrange* se ejecuta P veces utilizando la estrategia configurada en `GHC`. Por tanto, según la complejidad establecida en la Expresión 2.3, la generación de todos los planes de ejecución tendrá complejidad $P * T_{\text{rearrange}}(n)$. Esto corresponde a $O(P * n^2)$ cuando se utiliza la variante heurística pre-determinada y a $O(P * n^3)$ cuando se activa la variante óptima.

Para comparar los planes se incorporó la función `stmtTreeCost`, presentada en el Código 5.5. La función implementa la Expresión 1.1: asigna costo unitario a un statement, suma las composiciones secuenciales y adopta el máximo de las ramas aplicativas.

```

stmtTreeCost :: StmtTree a -> Cost
stmtTreeCost (StmtTreeOne _) = 1
stmtTreeCost (StmtTreeBind l r) =
  stmtTreeCost l + stmtTreeCost r
stmtTreeCost (StmtTreeApplicative ts) =
  case ts of
    [] -> 0
    _ -> Partial.maximum (map stmtTreeCost ts)

```

Código 5.5: Cálculo del costo estructural de un plan de ejecución.

La selección del mejor plan de ejecución utiliza `minimumBy` sobre la lista de árboles aplicando la función `stmtTreeCost`, escogiendo el primer mínimo, por lo que el orden estable de los reordenamientos válidos proporciona un desempate determinista. En el ejemplo probabilístico con Código 1.2, el orden original genera un árbol con costo 3 y los demás reordenamientos válidos obtienen árboles con costo 2; luego se selecciona el primero de ellos, asociado al plan de ejecución $(s_1 \mid s_3); (s_2 \mid s_4)$ como se muestra en la Tabla 1.2.

Finalmente, la función `stmtTreeToStmts` reconstruye la lista, genera nodos `ApplicativeStmt` y reincorpora el statement terminal. Desde ese punto la extensión deja de intervenir y, como resume la Expresión 5.1, el nodo AST renombrado continúa por el `Typechecker` y el `Desugarer` de forma normal.

Capítulo 6

Validación

El diseño y la implementación desarrollados en los capítulos anteriores permiten generar reordenamientos válidos de los statements de un bloque `do`, construir un plan de ejecución para cada uno y seleccionar una alternativa de costo estructural mínimo. Este capítulo establece cómo se evalúan experimentalmente esas capacidades. En particular, se busca determinar si los reordenamientos generados conservan el resultado observable de los programas considerados bajo una hipótesis explícita de conmutatividad, si la selección automática realizada por la extensión corresponde efectivamente a un plan de costo mínimo entre los candidatos evaluados y si la ruta conmutativa se activa únicamente en los bloques marcados explícitamente.

Estas propiedades y la condición de activación requieren observar aspectos distintos de la compilación. La preservación del resultado puede estudiarse mediante la ejecución y comparación de las variantes de cada programa asociadas a cada reordenamiento válido; en cambio, el reordenamiento elegido, el plan de ejecución construido, su costo y la ruta seguida por el Renamer no son visibles en la salida cuando la transformación conserva la semántica esperada. Por esta razón, la validación combina comparaciones de ejecución con instrumentación del Renamer, cuyas trazas permiten reconstruir los candidatos generados y las decisiones tomadas por la extensión.

Primeramente se presenta la metodología, incluyendo las métricas obtenidas de las trazas del Renamer y el flujo automatizado utilizado para compilar, ejecutar y contrastar cada reordenamiento válido. Luego se describen los corpus construidos con `Maybe` y `Dist.T Rational`, junto con un control negativo basado en `IO`. El capítulo termina delimitando el alcance de esta evaluación y sus principales limitaciones; la evidencia y resultados obtenidos se presentan por separado en el Capítulo 7.

6.1. Metodología

La metodología evalúa las dos propiedades establecidas por el tercer objetivo específico de la memoria y, adicionalmente, el requisito de activación explícita definido en el diseño de la solución. La primera corresponde a la preservación experimental del comportamiento

observable; la segunda, a la selección del plan de ejecución según el modelo de costo adoptado; y la tercera, a la activación de la ruta conmutativa únicamente cuando un bloque `do` contiene el marcador correspondiente. Estos aspectos se expresan mediante las siguientes preguntas de validación:

- PV1.** En los casos evaluados bajo una hipótesis explícita de conmutatividad, ¿todos los reordenamientos válidos generados preservan la salida observable y el código de salida del programa original?
- PV2.** ¿El plan de ejecución seleccionado posee el costo estructural mínimo entre todos los planes obtenidos a partir de los reordenamientos válidos generados?
- PV3.** Con `ApplicativeDo` habilitado, ¿la ruta conmutativa se activa únicamente en los bloques `do` marcados explícitamente mediante `CD.do` y permanece inactiva en bloques `do` ordinarios?

La primera pregunta estudia experimentalmente la preservación del comportamiento observable al activar la solución; no pretende demostrar equivalencia semántica para todo programa posible. La segunda evalúa la política de selección del mejor reordenamiento válido bajo el modelo de costo estructural adoptado. Cabe recalcar que un resultado correcto no implica que obligatoriamente el plan de ejecución resultante deba mejorar el costo de `ApplicativeDo`: es suficiente con que la extensión seleccione uno de los mínimos disponibles, incluso cuando el orden original ya sea mínimo o existan varios reordenamientos válidos candidatos empatados. La tercera pregunta evalúa el mecanismo de activación explícita de la extensión, independientemente de la preservación observable y de la selección por costo.

La igualdad de resultados observables y la validez estructural de los reordenamientos generados constituyen evidencias complementarias. La primera se comprueba mediante ejecución, mientras que la segunda se examina a partir de las trazas del `Renamer`, las que documentan las aristas del grafo de precedencia RAW, los reordenamientos válidos generados y la selección del plan de ejecución resultante. De esta forma, la validación no infiere la corrección de los reordenamientos únicamente a partir de que las ejecuciones produzcan la misma salida.

Para responder las preguntas **PV1** y **PV2**, primeramente es necesario tener un programa de `Haskell` para validar la extensión, este debe declarar solo un bloque `do` e imprimir en pantalla su expresión terminal que generada con `return`, de esta forma se puede obtener el resultado final para su posterior análisis. De este programa se materializarán cuatro variantes:

- **Programa original:** conserva el orden fuente y se compila sin `ApplicativeDo`. Esta variante constituye la referencia para la comparación de resultados observables.
- **ApplicativeDo:** activa `ApplicativeDo` sin el marcador de conmutatividad. Así se obtienen resultados asociados a solo la aplicación de este algoritmo.
- **Selección automática:** activa la extensión mediante un bloque `CD.do`. Las extensiones `QualifiedDo` y `ApplicativeDo` permanecen habilitadas y se obtiene el plan de ejecución de menor costo.

- **Selección forzada:** mantiene la extensión activa al igual que la variante anterior, pero fuerza la selección de un reordenamiento válido específico, para compilarlo y ejecutarlo de manera independiente.

La última variante se obtiene mediante la nueva opción de compilación

`-fado-reorder-candidate-n=<i>`.

Esta opción existe exclusivamente como instrumentación de validación. No activa la extensión, no genera candidatos adicionales ni modifica sus costos; una vez construidos los reordenamientos válidos, reemplaza la selección automática del plan de ejecución menos costoso por el plan de ejecución asociado al reordenamiento válido de índice i .

El flujo de validación de un programa comienza compilando las tres primeras variantes anteriormente mencionadas. La variante asociada a la selección automática del mejor plan se compila con la opción de compilación `-ddump-rn-trace`, lo que permite obtener desde la traza del Renamer la métrica `generated-permutations` y, con ella, el valor n , correspondiente a la cantidad de reordenamientos válidos generados. A continuación, el mismo archivo se recompila n veces con la opción de selección forzada. En cada compilación la extensión vuelve a construir el conjunto de reordenamientos candidatos, pero el árbol que continúa hacia las etapas posteriores de **GHC** corresponde al índice indicado en `-fado-reorder-candidate-n`. En total, para un bloque `do` que posea n reordenamientos válidos, se construyen y ejecutan $n + 3$ binarios. En la Figura 6.1 se muestra el flujo de validación de la solución, aplicada sobre un programa de **Haskell** específico.

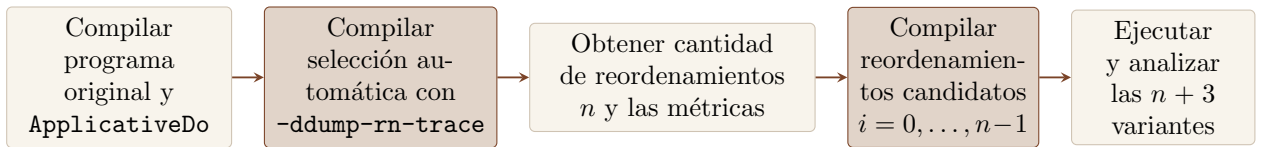


Figura 6.1: Flujo de validación de la solución para responder **PV1** y **PV2**.

La aplicación de la extensión sobre un programa considerado como conmutativo debe preservar el resultado observable. Precisamente por esta razón, una ejecución correcta no permite identificar qué reordenamientos válidos estudió el compilador, qué plan de ejecución construyó para cada uno ni cuál seleccionó. Para observar estas decisiones internas se incorporó al Renamer un esquema de trazas basado en la opción `-ddump-rn-trace` de **GHC**. Las compilaciones de la selección automática y de cada selección forzada activan esta opción; posteriormente, el flujo de validación procesa estas trazas y genera archivos legibles que contienen el grafo de precedencia RAW, los reordenamientos válidos generados y los árboles producidos por **ApplicativeDo** junto con sus costos asociados. Específicamente, de las trazas de la selección automática se obtienen las seis métricas de la Tabla 6.1.

Métrica	Interpretación
<code>minimum-cost-perm-index</code>	Índice, comenzando desde cero, del primer reordenamiento válido que alcanza el costo mínimo.
<code>original-cost</code>	Costo del plan de ejecución base completamente secuencial, con costo unitario por statement.
<code>applicative-do-cost</code>	Costo del plan de ejecución obtenido por <code>ApplicativeDo</code> al conservar el orden original.
<code>reorder-and-ado-minimum-cost</code>	Menor costo entre los planes de ejecución de cada reordenamiento válido generado.
<code>generated-permutations</code>	Cantidad de reordenamientos válidos generados, considerando el orden original dentro de ellos.
<code>minimum-cost-permutations</code>	Cantidad de reordenamientos válidos cuyos planes de ejecución tienen costo igual al mínimo encontrado.

Tabla 6.1: Métricas extraídas de las trazas del Renamer.

Las tres métricas de costo se calculan mediante el modelo estructural de la Expresión 1.1, caracterizando la secuencialidad y las agrupaciones aplicativas de los planes. Las métricas de cantidad caracterizan el espacio explorado y distinguen si el mínimo es único o si varios reordenamientos válidos poseen ese mismo costo.

Una vez terminadas las compilaciones, el flujo de validación ejecuta cada binario, captura su salida observable y registra su código de salida. Para responder la pregunta **PV1**, las variantes con `ApplicativeDo`, selección automática y selección forzada se comparan contra el programa original. La validación de un programa se considera exitosa solo si todos los códigos de salida y todas las salidas coinciden con el programa original. En el caso de utilizar mónadas probabilísticas, este programa debe normalizar su distribución resultante con `Dist.norm`, de modo que la comparación se realice sobre una representación canónica y no sobre diferencias internas de orden o duplicación. En caso de existir alguna salida que no coincida con lo esperado, se considera no validado.

Para responder la pregunta **PV2**, el análisis posterior extrae el costo registrado en la traza de cada reordenamiento candidato individual y vuelve a calcular el mínimo, su primer índice y la cantidad de empates. Estos valores se contrastan con el reordenamiento seleccionado automáticamente y con las métricas de la Tabla 6.1 emitidas por el Renamer; por tanto, estas no se utilizan como único criterio para aceptar la selección. Las cantidades y costos obtenidos también se comparan en cuyos casos estructurales con comportamientos esperados, por ejemplo grafos de precedencia sin aristas o cadenas de statements no reordenables.

Para responder la pregunta **PV3** se utiliza un flujo de validación reducido. A partir del programa en el que se desea validar la extensión se construyen tres variantes, cada una compilada con la opción `-ddump-rn-trace`, pues esta traza permite determinar si el Renamer ingresa en la ruta conmutativa. La primera habilita únicamente `ApplicativeDo` y conserva un bloque `do` ordinario. La segunda habilita además `QualifiedDo`, importa el módulo conmutativo y declara la instancia correspondiente, pero también conserva la notación ordinaria; por tanto, ninguna de estas dos variantes debe activar la ruta conmutativa. La tercera

mantiene la configuración de la segunda, pero escribe el bloque mediante `CD.do` y utiliza `CD.return` para su expresión terminal. Esta última variante sí debe activar la ruta y generar los reordenamientos válidos.

El mecanismo de compilación depende de las dependencias del programa que se desea validar. Los programas sin dependencias externas se compilan directamente con la versión modificada de `GHC`. Cuando el programa requiere paquetes externos administrados mediante `Cabal`, la compilación se realiza a través de esta herramienta, configurada para utilizar la misma versión modificada del compilador.

Ya definida la metodología de validación de la solución, la siguiente sección presenta los corpus de programas seleccionados para aplicar la validación, junto con su organización y los fenómenos cubiertos por sus casos de prueba.

6.2. Corpus de validación

La evaluación se organiza en dos corpus sintéticos positivos y un control negativo auxiliar. El primer corpus usa la mónada `Maybe` para estudiar el comportamiento estructural de la extensión en un contexto determinista. El segundo emplea la mónada probabilística `Dist.T Rational` para evaluar su aplicación en el dominio principal de la memoria: los programas probabilísticos escritos en `Haskell`. En conjunto reúnen 99 programas concretos: 47 basados en `Maybe` y 52 en distribuciones probabilísticas finitas. Su objetivo no es representar exhaustivamente todos los programas posibles, sino cubrir de manera sistemática las formas de dependencia, patrones de binders, clases de statements, comportamientos de costo y fenómenos semánticos que inciden directamente en la solución.

Cada programa se almacena de acuerdo con una jerarquía de familia conceptual, caso específico y variante interna. La variante identifica normalmente la cantidad de statements del bloque `do` del programa a validar, por ejemplo `03` o `04`; cuando existen varias variantes con la misma cantidad de statements se agrega un sufijo descriptivo, como `08-reorder-improves-ado`.

6.2.1. Familias compartidas

Los corpus comparten 42 variantes estructurales, adaptadas a las operaciones de cada mónada sin modificar la forma de dependencia que se pretende estudiar. La Tabla 6.2 resume su distribución. Como cada variante aparece una vez con `Maybe` y una vez con `Dist.T Rational`, este núcleo compartido aporta 84 de los 99 programas.

Familia	Variantes	Cobertura principal
<code>dependency-shapes</code>	14	Distintas formas del grafo de precedencia RAW: cadenas, grafo sin aristas, diamantes, fan-in y fan-out.
<code>binders</code>	11	Tuplas, listas, patrones anidados, wildcards, as-patterns y patrones lazy.
<code>let-statements</code>	4	Dependencias entre statements <code>let</code> y binders monádicos.
<code>body-stmts</code>	2	Statements sin binder, independientes y dependientes.
<code>shadowing</code>	4	Reutilización de nombres fuente con identidades renombradas distintas.
<code>cost-selection</code>	4	Empates de costo, mínimos únicos y órdenes originales mejorables.
<code>controls</code>	3	Activación e inactividad de la ruta conmutativa de la extensión.

Tabla 6.2: Familias y variantes compartidas por los dos corpus.

Estas familias aíslan el comportamiento del Renamer, la recolección de binders, el cálculo de variables libres locales, la construcción del grafo de precedencia RAW y la política de selección. Las variantes de `cost-selection` se retoman con código probabilístico en la Sección 7.3; las formas fan-in y fan-out permanecen dentro de la cobertura del corpus, aunque no se desarrollan como ejemplos visuales independientes.

6.2.2. Familias específicas

La familia `maybe-failure` aporta cinco variantes específicas al corpus de `Maybe`. Por su parte, `probability-distributions` aporta ocho variantes al corpus probabilístico y `probability-conditioning` agrega otras dos. La Tabla 6.3 resume esta distribución adicional al núcleo compartido.

Familia	Variantes	Cobertura principal
<code>maybe-failure</code>	5	<code>Nothing</code> , fallos de patrón y acciones que modelan guardas.
<code>probability-distributions</code>	8	Pesos, soportes dependientes, resultados repetidos y constructores de distribuciones.
<code>probability-conditioning</code>	2	Distribuciones imposibles y filtrado explícito de resultados.

Tabla 6.3: Familias específicas de cada corpus.

El corpus de `Maybe` entrega programas pequeños y deterministas para observar fallos y propiedades estructurales. El corpus de `Dist.T Rational`, por su parte, conecta la extensión con el dominio principal de la memoria. Todos sus programas imprimen una salida canónica

mediante `Dist.norm`, que agrupa resultados iguales y suma sus probabilidades. La comparación continúa siendo byte a byte, pero se realiza sobre distribuciones normalizadas para no confundir una diferencia de representación interna con una diferencia semántica.

6.2.3. Control negativo con IO

El caso con `IO` no se contabiliza como un tercer corpus. Se utiliza para mostrar por qué la preservación de dependencias de un reordenamiento válido no basta para una mónada arbitraria y por qué su equivalencia semántica exige una hipótesis explícita de conmutatividad. En `IO`, dos acciones que no comparten variables pueden tener efectos observables cuyo orden importa. Por ejemplo, dos impresiones independientes pueden producir salidas distintas si se intercambian.

El programa de control declara deliberadamente una instancia `CommutativeMonad IO` que no satisface la propiedad asumida y utiliza `CD.do` para activar la extensión. Por tanto, permite comprobar que el marcador constituye un contrato no verificado: si el programador declara como conmutativos efectos que no lo son, un reordenamiento estructuralmente válido puede cambiar la salida.

El control negativo también muestra por qué el compilador no debe inferir conmutatividad a partir de la sola ausencia de dependencias RAW. La extensión explora órdenes alternativos únicamente en bloques marcados explícitamente; para bloques `do` ordinarios, `ApplicativeDo` conserva su comportamiento y no se habilita la ruta conmutativa.

6.3. Alcance y limitaciones de la validación

Una vez explicada la metodología de validación y presentados los programas sobre los que se aplica, es necesario delimitar el alcance de la evidencia obtenida y explicitar las limitaciones del proceso. Estas consideraciones permiten interpretar adecuadamente los resultados presentados en el Capítulo 7 y evitar conclusiones que excedan los casos evaluados.

Primeramente, el alcance de la evaluación está condicionado por la naturaleza sintética de los 99 programas que componen los corpus. Estos fueron contruidos para aislar de manera controlada los fenómenos relevantes para la solución, pero no permiten estimar con qué frecuencia aparecen oportunidades de reordenamiento en aplicaciones reales. La cobertura tampoco abarca toda la sintaxis de `Haskell`: en particular, la implementación no incorpora lecturas que aparezcan exclusivamente dentro del patrón de un `BindStmt`, como puede ocurrir con `ViewPatterns`. De manera similar, los controles de activación consideran las tres configuraciones definidas en la metodología, pero no todas las combinaciones posibles de extensiones, importaciones y formas de calificar un bloque `do`.

Además de esta cobertura acotada, la evidencia asociada a **PV1** es de carácter observacional. La comparación de la salida capturada y del código de salida permite detectar diferencias en las ejecuciones de programas cerrados y finitos, pero no constituye una demostración formal de equivalencia semántica ni cubre divergencia o interacciones externas. En los programas

probabilísticos, `Dist.norm` agrupa resultados iguales y suma sus probabilidades antes de imprimir. Aunque esta normalización evita confundir diferencias de representación interna con diferencias semánticas, también pasa a formar parte del criterio observable adoptado.

Incluso cuando las ejecuciones producen resultados coincidentes, su interpretación depende de la conmutatividad declarada por el programador, pues `GHC` no demuestra esta propiedad. Si la declaración es incorrecta, preservar las dependencias RAW no basta para conservar el comportamiento. El control con `IO` muestra este límite: un reordenamiento puede ser estructuralmente válido y, aun así, modificar el orden de los efectos observables. Por tanto, los resultados positivos de `Maybe` y `Dist.T Rational` deben entenderse bajo la hipótesis explícita de conmutatividad adoptada por la memoria.

En cuanto a **PV2**, existe una limitación adicional porque las métricas, los candidatos y las decisiones provienen del mismo prototipo que se evalúa. Para evitar que el resumen interno provisto por el Renamer sea el único criterio de aceptación, el flujo compila cada candidato por separado, vuelve a calcular el mínimo y contrasta cantidades de reordenamientos que pueden determinarse independientemente. Aun así, no existe un oráculo externo que verifique todos los grafos y costos producidos.

Finalmente, aun cuando la selección sea correcta en los casos evaluados, la extensión enfrenta límites de escalabilidad y de interpretación del costo. La enumeración exhaustiva de ordenamientos topológicos puede crecer factorialmente en el peor caso, especialmente cuando todos los statements son independientes, por lo que su aplicación puede resultar impracticable en bloques sumamente grandes. A su vez, el modelo asigna un costo uniforme a cada statement y compara únicamente la estructura de los planes; no distingue operaciones baratas de costosas, no incorpora la sobrecarga de los combinadores ni mide paralelismo efectivo. En consecuencia, la evidencia permite validar la selección de un mínimo estructural bajo el modelo adoptado, pero no garantiza la escalabilidad de la extensión ni una mejora de rendimiento en tiempo de ejecución.

Capítulo 7

Resultados

La metodología y los corpus de validación presentados en el capítulo anterior establecen las condiciones bajo las cuales se evalúa experimentalmente la solución implementada. En este capítulo se muestran los resultados obtenidos al aplicar este procedimiento sobre los programas de cada corpus, siendo **Maybe** y **Dist.T Rational** las mónadas sobre las que se valida la solución. Primeramente se presentarán los resultados de forma agrupada, de modo que sea posible observar el comportamiento general de todas las variantes sometidas a validación.

Luego de mostrar los resultados generales, se analizan los casos destacables del corpus probabilístico. Para cada uno de ellos se presenta el código fuente que lo conforma junto con su grafo de precedencia RAW asociado, lo que permite relacionar las dependencias presentes en el código con la estructura construida por la extensión. Posteriormente se examinan los programas de la familia **cost-selection** para estudiar el comportamiento de la selección del mejor plan de ejecución ante distintos perfiles de costo. Finalmente, se describen los resultados del control negativo con **I0** y se discute el alcance de la evidencia obtenida.

7.1. Resultados generales

La metodología de validación de la solución mostrado en la Figura 6.1 se aplicó a 93 programas que no correspondían a controles de activación: 44 del corpus de **Maybe** y 49 del corpus probabilístico. En todos ellos, la variante de **ApplicativeDo**, la selección automática y cada selección forzada conservaron tanto el código de salida como la salida observable del programa original. En particular, los casos de **Dist.T Rational** produjeron exactamente la misma distribución después de aplicar **Dist.norm**, incluidos aquellos con pesos no uniformes, resultados repetidos, soportes dependientes y condicionamiento. Por tanto, todos los casos evaluados respondieron favorablemente **PV1** dentro del criterio observacional establecido.

En los mismos 93 programas, el costo de cada candidato compilado individualmente permitió recalcular el mínimo, su primer índice y la cantidad de empates. En todos los casos, estos valores coincidieron con las métricas emitidas por el Renamer y el candidato elegido

automáticamente correspondió al primer plan de costo mínimo entre los reordenamientos válidos generados. Este resultado responde favorablemente **PV2**. Además, como el orden fuente siempre pertenece al conjunto de candidatos, la selección automática nunca produjo un costo mayor que el obtenido por **ApplicativeDo** sobre ese mismo orden.

Los seis programas restantes asociados al control de activación de la extensión respondieron **PV3** y también produjeron el comportamiento esperado. En ambos corpus, los cuatro programas con bloques **do** ordinarios mantuvieron el marcador no activado, mostrado en el Renamer como `commutative-do = False`, incluso cuando **QualifiedDo** estaba habilitado, la interfaz conmutativa estaba importada y se había declarado la instancia correspondiente. Los dos bloques **do** escritos mediante **CD.do** produjeron `commutative-do = True` a travez de las trazas del Renamer, cada uno generando seis candidatos, incluyendo el orden original. En consecuencia, el uso completo de la solución requiere **ApplicativeDo**, **QualifiedDo**, la interfaz calificada, una instancia **CommutativeMonad** y el marcador local **CD.do** en el bloque que quiera ser declarado como conmutativo. La instancia expresa el contrato de conmutatividad y se comprueba durante el tipado, mientras que el marcador resoluble es la condición que distingue directamente ambas rutas dentro del Renamer.

La cantidad de casos que reducen estrictamente el costo de **ApplicativeDo** no se utiliza como indicador agregado de efectividad. La mayor parte del corpus fue construida para comprobar preservación observable, formación de dependencias, recolección de binders o selección correcta, y no para disponer deliberadamente un orden fuente desfavorable. Por ello, es esperable que una cadena de statements admita un único orden, que varios reordenamientos válidos tengan el mismo costo o que **ApplicativeDo** ya alcance el costo mínimo. Los cinco programas de cada corpus que obtuvieron mejores resultados respecto a **ApplicativeDo** fueron tres casos de la familia **cost-selection**, el caso asociado a dos cadenas de statements independientes y el caso en el que se usa **let** independiente.

Establecidos estos resultados globales, las secciones siguientes presentan casos representativos de las familias evaluadas. El objetivo es mostrar cómo los fenómenos presentes en el código fuente se reflejan en el grafo de precedencia RAW y cómo esa estructura determina los reordenamientos y planes que puede considerar la extensión.

7.2. Casos destacables

Ya declarados los resultados globales de los dos corpus evaluados, en esta sección se examinarán casos representativos de las familias de casos descritas en el capítulo anterior, las cuales se encuentran en las Tablas 6.2 y 6.3. Se utilizan las variantes de **Dist.T Rational** para mantener el análisis dentro del dominio probabilístico principal de la memoria; las variantes estructurales equivalentes de **Maybe** produjeron los mismos grafos y métricas.

7.2.1. Formas de dependencia

La familia `dependency-shapes` permite contrastar la cantidad de reordenamientos válidos generados por la solución con cantidades conocidas que se obtienen en función de la forma del grafo de precedencia RAW. Los cuatro casos siguientes abarcan desde una secuencia de statements no reordenable hasta un programa con un espacio de exploración de 70 reordenamientos válidos.

Cadena completa

El caso `chain-no-reorder` consiste en un bloque `do` en el que cada distribución utiliza el binder definido por el statement anterior. Esta forma produce una cadena completa de dependencias y fija el orden de todos los statements. A continuación, en el Código 7.1 y la Figura 7.1 se muestran el programa probabilístico asociado a este caso y el grafo de precedencia RAW construido por la extensión, respectivamente.

```
CD.do
  x1 <- Dist.uniform [1, 2]
  x2 <- Dist.uniform [x1 + 10, x1 + 20]
  x3 <- Dist.uniform [x2 + 100, x2 + 200]
  x4 <- Dist.uniform [x3 + 1000, x3 + 2000]
CD.return (x1, x2, x3, x4)
```

Código 7.1: Programa probabilístico asociado al caso `chain-no-reorder`.

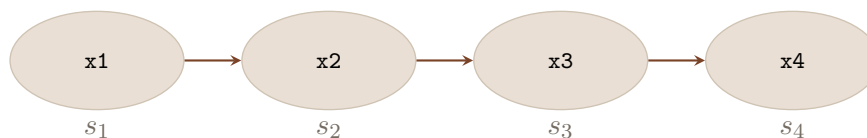


Figura 7.1: Grafo de precedencia RAW construido por la solución para el caso `chain-no-reorder`.

Como muestra la Figura 7.1, las aristas $s_1 \rightarrow s_2 \rightarrow s_3 \rightarrow s_4$ fijan por completo el orden de los cuatro statements. Por esta razón, la solución solo generó el candidato correspondiente al orden fuente y no encontró órdenes alternativos. La ejecución secuencial, `ApplicativeDo` y la solución obtuvieron un costo de 4. Este resultado confirma que las dependencias RAW restringen la generación de reordenamientos cuando todos los statements forman una cadena.

Diamante de seis statements

El caso `diamond` consiste en un bloque `do` que comienza con una distribución común, continúa con dos cadenas de distribuciones independientes entre sí y finaliza con una distribución que utiliza los resultados de ambas. Esta forma produce dos ramas que se separan a partir de `root` y convergen en `join`. A continuación, en el Código 7.2 y la Figura 7.2 se muestran el

programa probabilístico asociado a este caso y el grafo de precedencia RAW construido por la extensión, respectivamente.

```
CD.do
  root <- Dist.uniform [1, 2]
  a1 <- Dist.uniform [root + 10, root + 20]
  b1 <- Dist.uniform [root + 100, root + 200]
  a2 <- Dist.uniform [a1 + 1000, a1 + 2000]
  b2 <- Dist.uniform [b1 + 10000, b1 + 20000]
  join <- Dist.uniform [a2 + b2 + 100000, a2 + b2 + 200000]
CD.return (root, a1, a2, b1, b2, join)
```

Código 7.2: Programa probabilístico asociado al caso **diamond**.

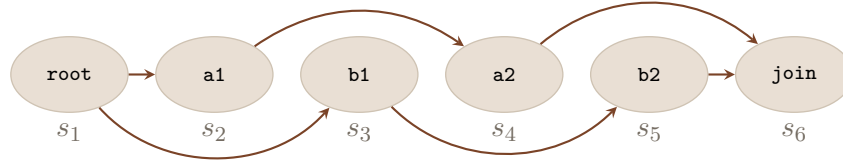


Figura 7.2: Grafo de precedencia RAW construido por la solución para el caso **diamond**.

La Figura 7.2 muestra un grafo que contiene las cadenas de aristas $s_1 \rightarrow s_2 \rightarrow s_4 \rightarrow s_6$ y $s_1 \rightarrow s_3 \rightarrow s_5 \rightarrow s_6$. Los statements de una rama pueden intercalarse con los de la otra siempre que cada cadena conserve su orden, por lo que el grafo admite seis órdenes válidos, cuatro de ellos con costo mínimo. La ejecución secuencial obtuvo un costo de 6, mientras que **ApplicativeDo** y la solución obtuvieron un costo de 4. Como **ApplicativeDo** ya construyó un plan mínimo sobre el orden fuente, la solución conservó ese primer candidato.

Grafo sin aristas

El caso **no-deps** consiste en un bloque **do** con cuatro distribuciones que no leen binders locales producidos por otros statements. Esta independencia produce un grafo de precedencia RAW sin aristas. A continuación, en el Código 7.3 y la Figura 7.3 se muestran el programa probabilístico asociado a este caso y el grafo de precedencia RAW construido por la extensión, respectivamente.

```
CD.do
  x1 <- Dist.uniform [1, 2]
  x2 <- Dist.uniform [10, 20]
  x3 <- Dist.uniform [100, 200]
  x4 <- Dist.uniform [1000, 2000]
CD.return (x1, x2, x3, x4)
```

Código 7.3: Programa probabilístico asociado al caso **no-deps**.



Figura 7.3: Grafo de precedencia RAW construido por la solución para el caso **no-deps**.

El grafo de la Figura 7.3 no contiene aristas, de modo que los cuatro statements pueden aparecer en cualquier orden. El grafo admite 24 órdenes válidos y todos alcanzaron el costo mínimo. La ejecución secuencial obtuvo un costo de 4, mientras que **ApplicativeDo** y la solución obtuvieron un costo de 1 al agrupar las cuatro distribuciones independientes. Como todos los candidatos tuvieron el mismo costo, la solución conservó el orden fuente.

Dos cadenas independientes

El caso **two-chains** consiste en un bloque **do** formado por dos cadenas independientes de cuatro distribuciones cada una. En el orden fuente, el último statement de cada cadena aparece separado de los tres anteriores, lo que dificulta que **ApplicativeDo** agrupe ambas cadenas completas. A continuación, en el Código 7.4 y la Figura 7.4 se muestran el programa probabilístico asociado a este caso y el grafo de precedencia RAW construido por la extensión, respectivamente.

```
CD.do
  a1 <- Dist.uniform [1, 2]
  a2 <- Dist.uniform [a1 + 100, a1 + 200]
  a3 <- Dist.uniform [a2 + 10000, a2 + 20000]
  b1 <- Dist.uniform [10, 20]
  b2 <- Dist.uniform [b1 + 1000, b1 + 2000]
  b3 <- Dist.uniform [b2 + 100000, b2 + 200000]
  a4 <- Dist.uniform [a3 + 1000000, a3 + 2000000]
  b4 <- Dist.uniform [b3 + 10000000, b3 + 20000000]
CD.return (a1, a2, a3, a4, b1, b2, b3, b4)
```

Código 7.4: Programa probabilístico asociado al caso **two-chains**.

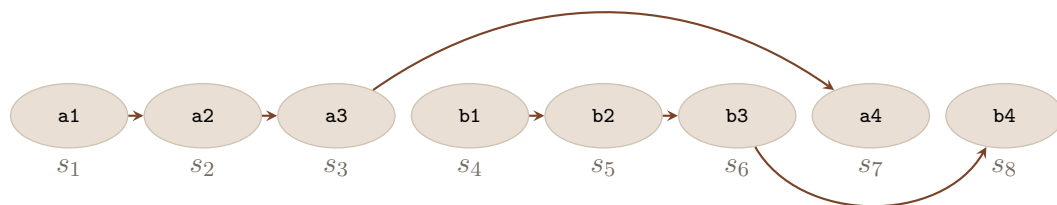


Figura 7.4: Grafo de precedencia RAW construido por la solución para el caso **two-chains**.

Como se muestra en la Figura 7.4, el grafo contiene la cadena de aristas $s_1 \rightarrow s_2 \rightarrow s_3 \rightarrow s_7$ junto con $s_4 \rightarrow s_5 \rightarrow s_6 \rightarrow s_8$. Como no existen dependencias entre ambas cadenas, sus

statements pueden intercalarse de 70 formas, 30 de las cuales alcanzaron el costo mínimo. La ejecución secuencial obtuvo un costo de 8 y **ApplicativeDo** redujo ese valor a 6. La solución reunió los statements de cada cadena y obtuvo un costo de 4, lo que muestra que un reordenamiento puede exponer agrupaciones aplicativas que no eran visibles en el orden fuente.

7.2.2. Dependencias introducidas por statements let

El caso **bind-depends-on-let** consiste en un bloque **do** cuyo primer statement define **x1** mediante **let**, mientras que el segundo utiliza ese identificador para construir una distribución y el tercero permanece independiente. Esta forma permite observar una dependencia RAW cuyo productor no es un binder monádico. A continuación, en el Código 7.5 y la Figura 7.5 se muestran el programa probabilístico asociado a este caso y el grafo de precedencia RAW construido por la extensión, respectivamente.

```
CD.do
  let x1 = 10
  x2 <- Dist.uniform [x1 + 1, x1 + 2]
  x3 <- Dist.uniform [100, 200]
CD.return (x1, x2, x3)
```

Código 7.5: Programa probabilístico asociado al caso **bind-depends-on-let**.

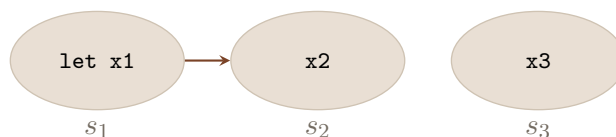


Figura 7.5: Grafo de precedencia RAW construido por la solución para el caso **bind-depends-on-let**.

La Figura 7.5 muestra un grafo de precedencia que contiene la arista $s_1 \rightarrow s_2$, cuyo identificador usado es **x1**, mientras que s_3 no presenta dependencias con los otros statements. El grafo admite tres órdenes válidos y todos alcanzaron el costo mínimo. La ejecución secuencial obtuvo un costo de 3, mientras que **ApplicativeDo** y la solución obtuvieron un costo de 2 al agrupar el tercer statement con la secuencia formada por los dos primeros. Este resultado confirma que la extensión considera las declaraciones **let** como productoras de binders al construir las precedencias RAW.

7.2.3. Binders definidos mediante patrones

El caso **tuple-pattern** consiste en un bloque **do** cuyo primer statement utiliza un patrón de tupla para definir simultáneamente los binders **x1** y **x2**. Los dos statements siguientes consumen estos binders por separado, lo que permite observar cómo la extensión recolecta

múltiples nombres desde un mismo patrón. A continuación, en el Código 7.6 y la Figura 7.6 se muestran el programa probabilístico asociado a este caso y el grafo de precedencia RAW construido por la extensión, respectivamente.

```
CD.do
  (x1, x2) <- Dist.uniform [(1, 10), (2, 20)]
  x3 <- Dist.uniform [x1 + 1, x1 + 2]
  x4 <- Dist.uniform [x2 + 1, x2 + 2]
  CD.return (x1, x2, x3, x4)
```

Código 7.6: Programa probabilístico asociado al caso `tuple-pattern`.

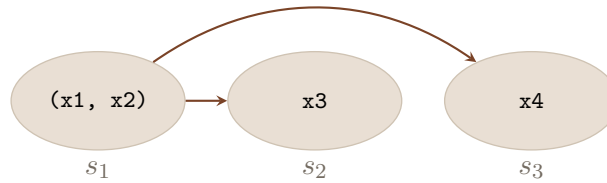


Figura 7.6: Grafo de precedencia RAW construido por la solución para el caso `tuple-pattern`.

El grafo de la Figura 7.6 contiene las aristas $s_1 \rightarrow s_2$ y $s_1 \rightarrow s_3$, cuyos identificadores consumidos son `x1` y `x2`, respectivamente. Como los dos `statements` posteriores no dependen entre sí, el grafo admite dos órdenes válidos y ambos alcanzaron el costo mínimo. La ejecución secuencial obtuvo un costo de 3, mientras que `ApplicativeDo` y la solución obtuvieron un costo de 2. Este resultado confirma que la extensión recolecta por separado los `binders` definidos dentro de un patrón de tupla.

7.2.4. Distribuciones con parámetros dependientes

El caso `choose-dependent` consiste en un bloque `do` en el que el valor de `x1` determina los pesos utilizados por `Dist.choose` para producir `x2`, mientras que `x3` permanece independiente. Esta forma permite observar una dependencia introducida dentro de una expresión condicional que modifica los parámetros de una distribución. A continuación, en el Código 7.7 y la Figura 7.7 se muestran el programa probabilístico asociado a este caso y el grafo de precedencia RAW construido por la extensión, respectivamente.

```
CD.do
  x1 <- Dist.uniform [1, 2]
  x2 <- if x1 == 1
    then Dist.choose (1 / 4) 10 20
    else Dist.choose (3 / 4) 10 20
  x3 <- Dist.uniform [100, 200]
  CD.return (x1, x2, x3)
```

Código 7.7: Programa probabilístico asociado al caso `choose-dependent`.



Figura 7.7: Grafo de precedencia RAW construido por la solución para el caso **choose-dependent**.

La condición que selecciona los pesos lee **x1**, por lo que la Figura 7.7 contiene la arista $s_1 \rightarrow s_2$, mientras que s_3 no presenta dependencias con los otros statements. El grafo admite tres órdenes válidos y todos alcanzaron el costo mínimo. La ejecución secuencial obtuvo un costo de 3, mientras que **ApplicativeDo** y la solución obtuvieron un costo de 2. Los tres órdenes conservaron la distribución normalizada y las probabilidades no uniformes definidas por cada rama, lo que muestra que la extensión detecta lecturas dentro de expresiones condicionales.

7.2.5. Condicionamiento de una distribución dependiente

El caso **filter-bind** consiste en un bloque **do** en el que el binder **x1** determina el soporte de la distribución entregada a **Dist.filter** en el segundo statement, mientras que el tercero permanece independiente. Esta forma representa el condicionamiento de una distribución que depende de un resultado producido previamente. A continuación, en el Código 7.8 y la Figura 7.8 se muestran el programa probabilístico asociado a este caso y el grafo de precedencia RAW construido por la extensión, respectivamente.

```

CD.do
  x1 <- Dist.uniform [1, 2]
  x2 <- Dist.filter even (Dist.uniform [x1, x1 + 1, x1 + 2])
  x3 <- Dist.uniform [100, 200]
  CD.return (x1, x2, x3)
  
```

Código 7.8: Programa probabilístico asociado al caso **filter-bind**.

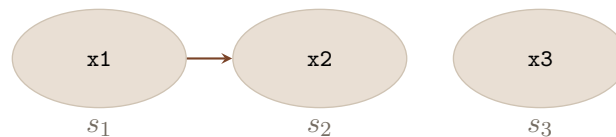


Figura 7.8: Grafo de precedencia RAW construido por la solución para el caso **filter-bind**.

La lectura de **x1** dentro de la distribución filtrada produce la arista $s_1 \rightarrow s_2$ de la Figura 7.8, mientras que s_3 no presenta dependencias con los otros statements. El grafo admite tres órdenes válidos y todos alcanzaron el costo mínimo. La ejecución secuencial obtuvo un costo de 3, mientras que **ApplicativeDo** y la solución obtuvieron un costo de 2. Los tres órdenes conservaron el soporte filtrado y sus probabilidades normalizadas, lo que confirma que la extensión mantiene la precedencia cuando una lectura aparece dentro de una operación de condicionamiento.

7.2.6. Identidades únicas frente a shadowing

El caso `read-after-rebind` consiste en un bloque `do` que reutiliza el identificador textual `x` en los dos primeros statements y luego lee ese identificador en el tercero para definir `y`. La segunda vinculación oculta a la primera, por lo que esta lectura debe depender del binder más reciente y no del `x` original. A continuación, en el Código 7.9 y la Figura 7.9 se muestran el programa probabilístico asociado a este caso y el grafo de precedencia RAW construido por la extensión, respectivamente.

```
CD.do
  x <- Dist.uniform [1, 2]
  x <- Dist.uniform [10, 20]
  y <- Dist.uniform [x + 100, x + 200]
CD.return (x, y)
```

Código 7.9: Programa probabilístico asociado al caso `read-after-rebind`.



Figura 7.9: Grafo de precedencia RAW construido por la solución para el caso `read-after-rebind` utilizando los identificadores ya renombrados por el Renamer.

La Figura 7.9 contiene la arista $s_2 \rightarrow s_3$, cuya lectura es `x_a1Y5`. No existe una arista desde s_1 , porque las identidades `x_a1Y4` y `x_a1Y5` representan binders distintos y la lectura del tercer statement fue asociada con el segundo. Por tanto, el primer statement permanece independiente de la cadena formada por los dos siguientes. El grafo admite tres órdenes válidos y todos alcanzaron el costo mínimo. La ejecución secuencial obtuvo un costo de 3, mientras que `ApplicativeDo` y la solución obtuvieron un costo de 2. Los tres órdenes conservaron la distribución normalizada, lo que confirma que la extensión respeta el shadowing y construye la dependencia RAW a partir de la identidad actualmente visible.

7.3. Selección por costo

La familia de casos `cost-selection` fue diseñada para observar la política de selección del plan de ejecución de menor costo bajo cuatro perfiles distintos. La Tabla 7.1 separa las tres métricas de costo y los dos conteos relevantes. *Sec* corresponde al costo base de ejecutar todos los statements en secuencia; *ADo*, al plan obtenido por `ApplicativeDo` sobre el orden fuente; y *Extensión*, al menor costo encontrado después de evaluar los reordenamientos válidos.

Caso	Sec	ADo	Extensión	Reordenamientos totales	Reordenamientos costo mínimo
all-same-cost	4	2	2	6	6
original-not-minimal	4	3	2	5	4
unique-minimum	5	5	4	5	1
many-minimums	6	4	3	20	14

Tabla 7.1: Costos y cantidades de reordenamientos de la familia `cost-selection`.

7.3.1. Todos los candidatos con el mismo costo

El caso `all-same-cost` consiste en un bloque `do` cuyo primer statement define `x1` y cuyos tres statements siguientes utilizan ese binder sin depender entre sí. Esta forma produce una raíz común con tres sucesores independientes y permite observar qué ocurre cuando todos los reordenamientos válidos tienen el mismo costo. A continuación, en el Código 7.10 y la Figura 7.10 se muestran el programa probabilístico asociado a este caso y el grafo de precedencia RAW construido por la extensión, respectivamente.

```

CD.do
  x1 <- Dist.uniform [1, 2]
  x2 <- Dist.uniform [x1 + 10, x1 + 20]
  x3 <- Dist.uniform [x1 + 100, x1 + 200]
  x4 <- Dist.uniform [x1 + 1000, x1 + 2000]
CD.return (x1, x2, x3, x4)

```

Código 7.10: Programa probabilístico asociado al caso `all-same-cost`.

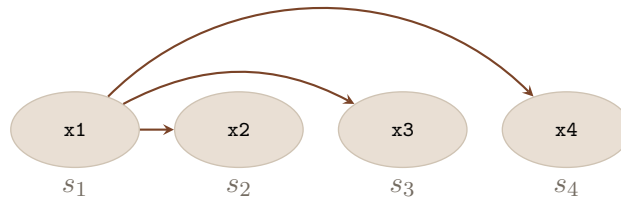


Figura 7.10: Grafo de precedencia RAW construido por la solución para el caso `all-same-cost`.

La Figura 7.10 contiene las aristas $s_1 \rightarrow s_2$, $s_1 \rightarrow s_3$ y $s_1 \rightarrow s_4$. Una vez ejecutado el primer statement, los otros tres pueden aparecer en cualquier orden, por lo que el grafo admite seis órdenes válidos y todos alcanzaron el costo mínimo. La ejecución secuencial obtuvo un costo de 4, mientras que `ApplicativeDo` y la solución obtuvieron un costo de 2 al agrupar los tres sucesores independientes. Como todos los candidatos tuvieron el mismo costo, la solución conservó de manera estable el orden fuente.

7.3.2. Orden original no mínimo

El caso `original-not-minimal` consiste en un bloque `do` en el que `x2` depende de `x1`, `x3` permanece independiente y `x4` utiliza tanto `x1` como `x3`. En el orden fuente, el statement que define `x2` aparece antes que la distribución independiente, lo que impide que `ApplicativeDo` exponga directamente las dos etapas aplicativas del grafo. Esta forma permite observar si la solución abandona el orden original cuando existe un reordenamiento de menor costo. A continuación, en el Código 7.11 y la Figura 7.11 se muestran el programa probabilístico asociado a este caso y el grafo de precedencia RAW construido por la extensión, respectivamente.

```
CD.do
  x1 <- Dist.uniform [1, 2]
  x2 <- Dist.uniform [x1 + 10, x1 + 20]
  x3 <- Dist.uniform [100, 200]
  x4 <- Dist.uniform [x1 + x3, x1 + x3 + 1]
CD.return (x1, x2, x3, x4)
```

Código 7.11: Programa probabilístico asociado al caso `original-not-minimal`.

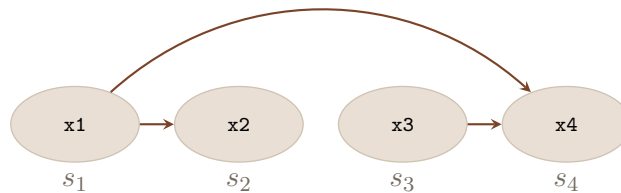


Figura 7.11: Grafo de precedencia RAW construido por la solución para el caso `original-not-minimal`.

La Figura 7.11 contiene las aristas $s_1 \rightarrow s_2$, $s_1 \rightarrow s_4$ y $s_3 \rightarrow s_4$. El grafo admite cinco órdenes válidos. La ejecución secuencial obtuvo un costo de 4 y `ApplicativeDo` obtuvo un costo de 3 sobre el orden fuente, que fue el único candidato con ese valor. Los otros cuatro órdenes alcanzaron un costo de 2; la solución seleccionó el primero de ellos, agrupando inicialmente los statements independientes s_1 y s_3 y luego los sucesores s_2 y s_4 . Este resultado confirma que la política abandona el orden original cuando existe una alternativa de menor costo.

7.3.3. Mínimo único

El caso `unique-minimum` consiste en un bloque `do` que combina una cadena de cuatro statements con un statement independiente que utiliza un patrón de tupla. Aunque este último no presenta dependencias RAW, su patrón potencialmente estricto hace que `ApplicativeDo` conserve la secuencia con el statement siguiente cuando aparece antes del final del bloque. Esta forma produce un único reordenamiento de costo mínimo. A continuación, en el Código 7.12 y la Figura 7.12 se muestran el programa probabilístico asociado a este caso y el grafo de precedencia RAW construido por la extensión, respectivamente.

```

CD.do
  x1 <- Dist.uniform [1, 2]
  x2 <- Dist.uniform [x1 + 10, x1 + 20]
  x3 <- Dist.uniform [x2 + 100, x2 + 200]
  (x4, _) <- Dist.uniform [(40, 0), (50, 0)]
  x5 <- Dist.uniform [x3 + 1000, x3 + 2000]
CD.return (x1, x2, x3, x4, x5)

```

Código 7.12: Programa probabilístico asociado al caso **unique-minimum**.

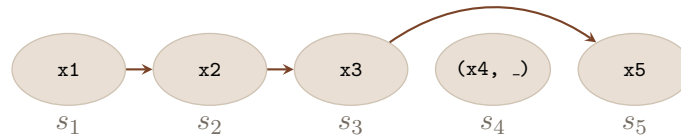


Figura 7.12: Grafo de precedencia RAW construido por la solución para el caso **unique-minimum**.

La Figura 7.12 contiene la cadena de aristas $s_1 \rightarrow s_2 \rightarrow s_3 \rightarrow s_5$, mientras que s_4 permanece aislado porque su expresión no lee binders locales y **x4** no es utilizado por otro statement del grafo. Los cinco órdenes válidos corresponden a las posiciones en que puede insertarse s_4 respecto de la cadena. La ejecución secuencial y **ApplicativeDo** obtuvieron un costo de 5 sobre el orden fuente. Cuatro candidatos conservaron ese costo y solo el orden que ubica s_4 después de s_5 alcanzó un costo de 4, por lo que la solución seleccionó ese mínimo único y agrupó la cadena completa con el statement independiente.

7.3.4. Varios candidatos mínimos

El caso **many-minimums** consiste en un bloque **do** formado por dos cadenas independientes de tres statements cada una. El orden fuente comienza la primera cadena, desarrolla por completo la segunda y deja al final el último statement de la primera, de modo que distintos intercalados pueden exponer agrupaciones aplicativas equivalentes. Esta forma permite observar cómo la solución resuelve un empate entre varios reordenamientos válidos de costo mínimo. A continuación, en el Código 7.13 y la Figura 7.13 se muestran el programa probabilístico asociado a este caso y el grafo de precedencia RAW construido por la extensión, respectivamente.

```

CD.do
  x1 <- Dist.uniform [1, 2]
  x2 <- Dist.uniform [x1 + 10, x1 + 20]
  x3 <- Dist.uniform [100, 200]
  x4 <- Dist.uniform [x3 + 1000, x3 + 2000]
  x5 <- Dist.uniform [x4 + 10000, x4 + 20000]
  x6 <- Dist.uniform [x2 + 100000, x2 + 200000]
CD.return (x1, x2, x3, x4, x5, x6)

```

Código 7.13: Programa probabilístico asociado al caso `many-minimums`.

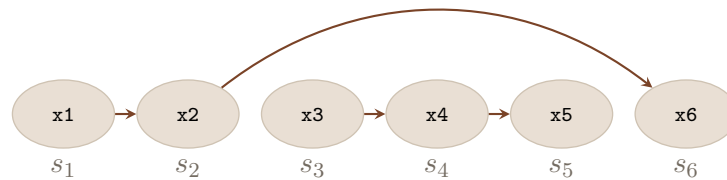


Figura 7.13: Grafo de precedencia RAW construido por la solución para el caso `many-minimums`.

La Figura 7.13 contiene las cadenas de aristas $s_1 \rightarrow s_2 \rightarrow s_6$ y $s_3 \rightarrow s_4 \rightarrow s_5$, sin dependencias entre ellas. Sus intercalados producen 20 órdenes válidos: 14 alcanzan el costo mínimo de 3 y los otros seis obtienen un costo de 4. La ejecución secuencial obtuvo un costo de 6 y `ApplicativeDo` redujo ese valor a 4 sobre el orden fuente. La solución obtuvo un costo de 3 al disponer cada cadena como una rama completa y seleccionó de manera estable el primero de los 14 candidatos mínimos. En conjunto, los cuatro perfiles confirman que la política funciona con mínimos únicos o empatados y no presupone que todos los casos deban mejorar el plan de `ApplicativeDo`.

7.4. Control negativo con IO

La evidencia positiva presentada en las secciones anteriores se obtuvo sobre programas evaluados bajo una hipótesis explícita de conmutatividad. Para contrastar ese escenario, esta sección retoma el control negativo con `IO` definido en el capítulo anterior. Su propósito es mostrar que un reordenamiento puede respetar todas las dependencias RAW y, aun así, modificar el comportamiento observable cuando la mónada utilizada no es conmutativa.

El programa sobre el que se aplicará el control negativo de la solución declara deliberadamente una instancia `CommutativeMonad IO` que no satisface la propiedad que esta clase representa. Dentro del bloque `CD.do`, el primer statement imprime `A` y produce el valor 1, mientras que el segundo imprime `B` y produce el valor 2. Ninguna acción lee el binder definido por la otra, pero ambas realizan efectos visibles sobre la salida estándar. A continuación, en el Código 7.14 y la Figura 7.14 se muestran el programa asociado a este control y el grafo de precedencia RAW construido por la extensión, respectivamente.

```

instance CD.CommutativeMonad IO

program :: IO Int
program = CD.do
  x <- putStrLn "A" >> pure 1
  y <- putStrLn "B" >> pure 2
  CD.return (x + y)

```

Código 7.14: Programa asociado al control negativo con IO.



Figura 7.14: Grafo de precedencia RAW construido por la solución para el control negativo con IO.

El grafo de la Figura 7.14 no contiene aristas, ya que ninguno de los dos statements lee el binder producido por el otro. Por esta razón, el grafo admite los órdenes $[1,2]$ y $[2,1]$, y ambos candidatos alcanzaron el costo mínimo. La ejecución secuencial obtuvo un costo de 2, mientras que `ApplicativeDo` y la solución obtuvieron un costo de 1. Como los dos candidatos empataron, la selección estable escogió el candidato de índice 0 y conservó el orden fuente $[1,2]$.

Desde la perspectiva del grafo y del modelo de costo, los dos órdenes son indistinguibles. Sin embargo, la ausencia de dependencias RAW no asegura que los efectos de las acciones puedan intercambiarse. Además del candidato seleccionado, la validación exhaustiva compiló y ejecutó el candidato forzado de índice 1, correspondiente al orden $[2,1]$. Ambos terminaron con código de salida cero y retornaron el entero 3, pero produjeron las secuencias observables resumidas en la Tabla 7.2.

Orden	Primera línea	Segunda línea	Resultado
$[1,2]$	A	B	3
$[2,1]$	B	A	3

Tabla 7.2: Salidas observables producidas por los dos órdenes del control negativo con IO.

Como muestra la Tabla 7.2, el orden fuente imprimió primero A y luego B, mientras que el candidato invertido produjo ambas líneas en el orden contrario. La comparación con el programa original detectó esta diferencia y terminó la validación con fracaso. Este resultado es el desenlace esperado del control y no representa un error de la implementación: confirma que un ordenamiento topológico preserva las dependencias locales de datos, pero no garantiza que los efectos de una mónada arbitraria puedan intercambiarse. La instancia `CommutativeMonad` expresa un contrato semántico que el compilador no demuestra, por lo que una declaración incorrecta puede habilitar transformaciones estructuralmente válidas que alteran el comportamiento observable.

7.5. Discusión

Respecto de **PV1**, los 93 programas sometidos al flujo de validación de la solución mostrado en la Figura 6.1 conservaron el código de salida y la salida observable en **ApplicativeDo**, en la selección automática y en cada reordenamiento válido forzado. La coincidencia abarcó las familias estructurales de **Maybe** y **Dist.T Rational**, además de pesos no uniformes, soportes dependientes y condicionamiento probabilístico. Los casos analizados en la Sección 7.2 muestran que esta preservación se mantuvo ante grafos y construcciones sintácticas diferentes. La evidencia permite responder favorablemente la pregunta para los programas estudiados bajo la hipótesis explícita de conmutatividad, pero continúa siendo observacional y no constituye una demostración de equivalencia semántica para todo programa posible.

Respecto de **PV2**, en los mismos 93 programas la selección automática coincidió con el primer mínimo recalculado desde los costos de los candidatos compilados individualmente. La Tabla 7.1 y sus cuatro casos muestran que la política se comportó correctamente cuando todos los candidatos empataron, cuando el orden original no fue mínimo, cuando existió un único mínimo y cuando varios reordenamientos compartieron el menor costo. Que un programa no reduzca el costo de **ApplicativeDo** no representa un fallo: PV2 exige seleccionar un mínimo entre los planes generados, no producir necesariamente una mejora estricta. Como el orden original forma parte del espacio explorado, el costo elegido tampoco puede superar el plan que **ApplicativeDo** obtiene sobre ese orden.

La cantidad de mejoras estrictas observadas no permite estimar la utilidad de la extensión en programas reales. Los corpus son sintéticos y la mayoría de sus casos fue diseñada para aislar preservación observable, dependencias, binders o empates, no para construir un orden fuente desfavorable. Las cinco variantes que sí mejoraron fueron replicadas con ambas mónadas y preparadas precisamente para admitir un plan menor. El resultado relevante no es su proporción dentro del corpus, sino que la extensión reconoció la alternativa mínima cuando existía y mantuvo un mínimo cuando no había una alternativa mejor.

Respecto de **PV3**, los seis controles distinguieron correctamente las rutas estándar y conmutativa. Con **ApplicativeDo** habilitado, los cuatro bloques **do** ordinarios mantuvieron en las trazas del Renamer **commutative-do = False**, aun en presencia de **QualifiedDo**, la importación de la interfaz y la instancia correspondiente. Los dos bloques **CD.do** produjeron **commutative-do = True** y generaron seis candidatos cada uno, uno original y cinco alternativos. Estos resultados confirman la activación local mediante el marcador dentro de las configuraciones evaluadas. La instancia **CommutativeMonad** sigue siendo necesaria para utilizar las operaciones calificadas, pero expresa una restricción comprobada durante el tipado y no constituye por sí sola el disparador del Renamer.

Los grafos presentados también muestran que el tamaño del espacio de búsqueda y la calidad del plan son propiedades relacionadas, pero diferentes. Una cadena completa produjo un único candidato; un grafo vacío de cuatro vértices produjo 24; y dos cadenas independientes de longitud cuatro produjeron 70. Sin embargo, el diamante mantuvo el mínimo que **ApplicativeDo** ya alcanzaba, mientras el orden desfavorable de las dos cadenas permitió reducir el costo de 6 a 4. Del mismo modo, las familias de **let**, binders, distribuciones dependientes, condicionamiento y *shadowing* confirmaron que las aristas se forman a partir

de las identidades y lecturas locales detectadas por el Renamer, no solo a partir de binders monádicos simples.

Finalmente, el control con `I0` de la Sección 7.4 separa la validez estructural de la equivalencia semántica. El candidato invertido preservó el grafo RAW vacío y retornó el mismo entero, pero cambió el orden de los efectos visibles. Su divergencia no contradice los resultados positivos, sino que confirma el límite previsto por el diseño: las precedencias RAW protegen el flujo local de datos, mientras la posibilidad de intercambiar efectos depende de la conmutatividad declarada. Asimismo, las reducciones informadas corresponden al modelo estructural uniforme y no a mediciones de tiempo de ejecución. Bajo estas condiciones, los resultados responden favorablemente las tres preguntas de validación dentro del alcance delimitado en la Sección 6.3.

Capítulo 8

Conclusión

8.1. Resumen del Trabajo

Esta memoria presentó una extensión experimental de `ApplicativeDo` en `GHC` para explorar reordenamientos válidos de statements en bloques `do` marcados como conmutativos. El punto de partida fue una limitación del algoritmo existente: `ApplicativeDo` puede descubrir independencia y construir planes applicative, pero preserva el orden relativo original de los statements. Esta decisión es correcta para mónadas arbitrarias, aunque puede ser conservadora cuando la mónada permite conmutar subcómputos independientes.

La solución implementada agrega un mecanismo explícito de opt-in sintáctico. Los bloques escritos como `CD.do` se reconocen usando un marcador exportado por el módulo conmutativo experimental. Para esos bloques, el Renamer construye un grafo de precedencia RAW, enumera los ordenamientos topológicos del grafo, interpreta cada uno como un reordenamiento válido, evalúa cada reordenamiento válido con el algoritmo existente de `ApplicativeDo` y selecciona el candidato de menor costo.

La validación se apoyó en corpus sintéticos para `Maybe` y para la mónada probabilística `Dist.T Rational`, además de un control negativo con `IO`. El ejemplo guía académico mostró una mejora estricta de costo: ejecución secuencial de costo 4, `ApplicativeDo` normal de costo 3 y un candidato seleccionado de costo 2 obtenido a partir de un reordenamiento válido. Bajo la hipótesis explícita de conmutatividad, los cinco reordenamientos válidos generados conservaron exactamente la distribución normalizada del programa original y la validación terminó con `RESULT: OK`.

8.2. Cumplimiento de Objetivos

La Tabla siguiente resume el cumplimiento de los objetivos definidos en la Introducción.

Objetivo	Resultado
Caracterizar los reordenamientos válidos según las dependencias locales del bloque.	Se definió un grafo de precedencia RAW y se estableció que sus ordenamientos topológicos corresponden a las secuencias completas que preservan dichas dependencias.
Ampliar el conjunto de planes de ejecución considerado por <code>ApplicativeDo</code> bajo una hipótesis explícita de conmutatividad.	Se integraron el reconocimiento de la marca explícita, la generación de reordenamientos válidos, su evaluación con <code>ApplicativeDo</code> para formar candidatos y la selección automática del primer candidato de costo mínimo.
Comprobar experimentalmente la preservación del resultado observable y la selección de un plan de costo estructural mínimo.	Se probaron los corpus sintéticos de <code>Maybe</code> y <code>Dist.T Rational</code> con logs reproducibles de costos, grafos y salidas; además, el control negativo con <code>IO</code> mostró el riesgo de una declaración de conmutatividad incorrecta.

En conjunto, los objetivos quedaron cubiertos por la modificación de `GHC`, los módulos agregados, los dos corpus sintéticos, el control negativo con `IO`, los scripts de validación y la documentación del repositorio.

8.3. Limitaciones

La primera limitación es semántica: `GHC` no prueba que una mónada sea conmutativa. El mecanismo implementado se basa en una declaración explícita del programador mediante `CD.do`. Si esa declaración es incorrecta, el compilador puede aceptar reordenamientos válidos respecto de las dependencias locales que cambien efectos observables, como ilustra el control con `IO`.

La segunda limitación es combinatoria. Enumerar ordenamientos topológicos puede ser costoso cuando el bloque tiene muchos statements independientes. El prototipo permite estudiar exhaustivamente casos pequeños y medianos, pero requiere estrategias adicionales para escalar a programas de mayor tamaño.

La tercera limitación está en la validación. Comparar salidas observables entrega evidencia práctica para los casos evaluados, pero no equivale a una prueba formal de preservación semántica para todos los programas. En el corpus probabilístico, la normalización de distribuciones reduce diferencias representacionales, pero también fija una convención específica de comparación.

Finalmente, el modelo de costos es uniforme por statement. Esta simplificación permite comparar estructura de planes de manera reproducible, pero no distingue entre statements baratos y costosos. Por lo tanto, el menor costo abstracto no necesariamente equivale al menor tiempo de ejecución real.

8.4. Trabajo Futuro

Una línea directa de trabajo futuro es refinar el modelo de costos. La implementación actual usa costo uniforme por statement, siguiendo un criterio simple y reproducible. Una extensión posterior podría permitir costos anotados o inferidos, por ejemplo `low`, `medium` y `high`, interpretados como costos 1, 2 y 3. Con ello, dos planes con el mismo número de niveles applicative podrían diferenciarse por el peso relativo de sus statements.

```
defaultExample :: Maybe Int
defaultExample = CD.do
  x1 <- Just 5           -- high
  x2 <- Just 10          -- low
  x3 <- Just (x1 + 15)   -- medium
  x4 <- Just (x2 + 20)   -- high
  CD.return (x3 + x4)
```

Otra línea es reducir el espacio de búsqueda. En lugar de enumerar todos los ordenamientos topológicos, el compilador podría usar heurísticas, límites configurables o búsqueda guiada por costo parcial. Esto permitiría aplicar la idea a bloques más grandes sin pagar siempre el costo de una enumeración exhaustiva.

También queda abierta una evaluación de rendimiento real. La memoria compara planes usando un costo abstracto heredado de `ApplicativeDo`; un trabajo posterior podría medir tiempos de compilación, tiempos de ejecución y efectos sobre backends o librerías que aprovechen paralelismo applicative.

Finalmente, sería valioso integrar casos de esta extensión en una testsuite más cercana a `GHC`, mejorar las trazas para inspección humana y avanzar hacia una formalización más completa de la preservación semántica bajo la hipótesis de conmutatividad.

8.5. Cierre

El trabajo muestra una vía concreta para ampliar `ApplicativeDo` sin abandonar el compilador real ni la sintaxis habitual de `Haskell`. Bajo una hipótesis explícita de conmutatividad, el compilador puede evaluar todos los reordenamientos válidos, incluido el orden escrito por el programador; por definición, todos preservan las dependencias locales de variables.

El aporte principal no es solo el algoritmo de generación de reordenamientos válidos, sino su integración experimental en `GHC` con trazabilidad: módulos agregados, parches, corpus, scripts y logs reproducibles. Esto permite estudiar la idea de manera concreta, observar sus beneficios en casos controlados y delimitar sus riesgos.

En síntesis, la memoria entrega evidencia experimental de que el espacio de planes de `ApplicativeDo` puede ampliarse para mónadas declaradas como conmutativas, obteniendo mejoras de costo sin modificar la semántica observable en los casos sintéticos evaluados.

Bibliografía

- [1] Andrew W. Appel. SSA is functional programming. *ACM SIGPLAN Notices*, 33(4):17–20, 1998.
- [2] A. J. Bernstein. Analysis of programs for parallel processing. *IEEE Transactions on Electronic Computers*, EC-15(5):757–763, 1966.
- [3] Esteban Romero Berríos. Repositorio experimental para el estudio de applicative-do y reordenamiento de statements en ghc, 2026. GitHub repository.
- [4] Alan L. Davis and Robert M. Keller. Data flow program graphs. *Computer*, 15(2):26–41, 1982.
- [5] M. Anton Ertl and Andreas Krall. Removing anti dependences by repairing. In *Compiler Construction*, volume 1060 of *Lecture Notes in Computer Science*, pages 33–43. Springer, 1996.
- [6] Martin Erwig and Steve Kollmansberger. Functional pearls: Probabilistic functional programming in haskell. *Journal of Functional Programming*, 16(1):21–34, 2006.
- [7] Martin Erwig and Steve Kollmansberger. *Numeric.Probability.Distribution: Deterministic and Probabilistic Values*. Hackage, 2026. Documentación del paquete `probability`, versión 0.2.9.1.
- [8] Kai Feng, Jeremy Singer, and Angelos K. Marnierides. Fuzzrducc: Fuzzing with reconstructed def-use chain coverage, 2025.
- [9] Zoubin Ghahramani. Probabilistic machine learning and artificial intelligence. *Nature*, 521(7553):452–459, 2015.
- [10] GHC Contributors. Commit 8ecf6d8f7dfce9e5b1844cd196f83f00f3b6b879, 2015. GitHub commit.
- [11] GHC Contributors. GHC renamer source code. Código fuente oficial de GHC, 2026. Commit 0b36e96; <https://gitlab.haskell.org/ghc/ghc/-/tree/0b36e96/compiler/GHC>.
- [12] GHC Contributors. Testsuite de la extensión applicivedo en ghc, 2026. Directorio `testsuite/tests/ado` del repositorio oficial de GHC.
- [13] The Glasgow Haskell Compiler Team. *GHC User’s Guide: Applicative do-notation*, 2026. Documentación oficial de GHC.

- [14] The Glasgow Haskell Compiler Team. *GHC User's Guide: Qualified do-notation*, 2026. Documentación oficial de GHC.
- [15] Andrew D. Gordon, Thomas A. Henzinger, Aditya V. Nori, and Sriram K. Rajamani. Probabilistic programming. In *Future of Software Engineering Proceedings*, pages 167–181. ACM, 2014.
- [16] Mary Jean Harrold and Mary Lou Soffa. Efficient computation of interprocedural definition-use chains. *ACM Transactions on Programming Languages and Systems*, 16(2):175–204, 1994.
- [17] Bart Jacobs. From probability monads to commutative effectuses. *Journal of Logical and Algebraic Methods in Programming*, 94:200–237, 2018.
- [18] Arthur B. Kahn. Topological sorting of large networks. *Communications of the ACM*, 5(11):558–562, 1962.
- [19] Donald E. Knuth and Jayme L. Szwarcfiter. A structured program to generate all topological sorting arrangements. *Information Processing Letters*, 2(6):153–157, 1974.
- [20] Anders Kock. Commutative monads as a theory of distributions. *Theory and Applications of Categories*, 26(4):97–131, 2012.
- [21] David J. Kuck, Robert H. Kuhn, David A. Padua, Bruce Leasure, and Michael Wolfe. Dependence graphs and compiler optimizations. In *Proceedings of the 8th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, pages 207–218. ACM, 1981.
- [22] Simon Marlow. *Haskell 2010 Language Report*. Haskell Community, 2010. Informe oficial de la especificación del lenguaje.
- [23] Simon Marlow. *ado*, 2015. GitHub repository.
- [24] Simon Marlow, Louis Brandy, Jonathan Coens, and Jon Purdy. There is no fork: An abstraction for efficient, concurrent, and concise data access. In *Proceedings of the 19th ACM SIGPLAN International Conference on Functional Programming*, pages 325–337. ACM, 2014.
- [25] Simon Marlow, Simon Peyton Jones, Edward Kmett, and Andrey Mokhov. Desugaring Haskell's do-notation into applicative operations. In *Proceedings of the 9th International Symposium on Haskell*, pages 92–104. ACM, 2016.
- [26] Simon Marlow, Ryan Newton, and Simon Peyton Jones. A monad for deterministic parallelism. In *Proceedings of the 4th ACM SIGPLAN Symposium on Haskell*, pages 71–82. ACM, 2011.
- [27] Conor McBride and Ross Paterson. Applicative programming with effects. *Journal of Functional Programming*, 18(1):1–13, 2008.
- [28] Andrey Mokhov, Georgy Lukyanov, Simon Marlow, and Jeremie Dimino. Selective applicative functors. *Proceedings of the ACM on Programming Languages*, 3(ICFP):1–29, 2019.

- [29] Justin Pombrio. *Resugaring: Lifting Evaluation Sequences through Syntactic Sugar*. PhD thesis, Brown University, 2018.
- [30] Vaughan Pratt. Modeling concurrency with partial orders. *International Journal of Parallel Programming*, 15(1):33–71, 1986.
- [31] Adam Ścibior, Zoubin Ghahramani, and Andrew D. Gordon. Practical probabilistic programming with monads. In *Proceedings of the 8th ACM SIGPLAN Symposium on Haskell*, pages 165–176. ACM, 2015.
- [32] The Glasgow Haskell Compiler Team. GHC.Internal.Base: Basic definitions. Código fuente oficial de GHC, 2026. Commit 0b36e96, <https://gitlab.haskell.org/ghc/ghc/-/blob/0b36e96cb93db71f201aaa055c4a90b75a8110ef/libraries/ghc-internal/src/GHC/Internal/Base.hs>.
- [33] Philip Wadler. Monads for functional programming. In *Advanced Functional Programming*, volume 925 of *Lecture Notes in Computer Science*, pages 24–52. Springer, 1995.
- [34] Brent Yorgey. The typeclassopedia. *The Monad.Reader*, 13:17–68, 2009.

Apéndice A

Anexos Técnicos

A.1. Reproducibilidad

Esta sección de anexo debe documentar el entorno reproducible. Existe para que el cuerpo principal no se sobrecargue con detalles de Dev Container, toolchain y build de GHC. Lo ideal es incluir comandos, versiones y referencias a `docs/toolchain.md`.

Para completarla, se deben listar versiones de GHC de bootstrap, Cabal, Alex, Happy y comandos Makefile relevantes. También conviene explicar la ruta esperada del binario modificado de GHC dentro del contenedor.

A.2. Pipeline de Validación

Esta sección de anexo debe describir con detalle los comandos usados para validar. Existe para que Validación pueda concentrarse en el diseño experimental y Resultados en la evidencia obtenida, dejando aquí la información operativa. Lo ideal es incluir ejemplos de `make test-ghc` y `make test-cabal`.

Para completarla, se debe documentar la estructura de entrada bajo `experiments/` y la estructura de salida bajo `tests/`. También conviene explicar la diferencia entre backend GHC directo y backend Cabal para dependencias externas como `probability`. El ejemplo guía debe presentarse como un recorrido completo del backend Cabal, desde `experiments/main-example/main.hs` hasta los resultados de `tests/main-example`.

A.3. Logs y Trazas

Esta sección de anexo debe reunir ejemplos de logs representativos. Existe porque las trazas completas del Renamer son demasiado densas para el cuerpo principal, pero son im-

portantes para reproducibilidad y depuración. Lo ideal es incluir fragmentos de `summary.log`, `optimal-reorder.log` y `semantic-validation.log`.

Para completarla, la fuente principal debe ser `tests/main-example: summary.log` resume los costos, `graph.log` presenta las dependencias, `logs/original_ado.log` muestra el plan sin reordenamiento, `logs/optimal-reorder.log` registra el candidato elegido y `semantic-validation.log` confirma la preservación de la salida. También conviene incluir un caso estructural de `Maybe` y el control negativo con `IO` para mostrar comportamientos complementarios.

A.4. Corpus

Esta sección de anexo debe contener tablas completas de los corpus. Existe para que Validación pueda describir sus familias y Resultados pueda presentar solo casos representativos sin perder trazabilidad del conjunto completo. Lo ideal es basarse en `experiments/maybe-monad/cases.md` y `experiments/probability-monad/cases.md`.

Para completarla, se deben copiar o sintetizar las tablas de familias, casos y variantes implementadas para ambos corpus sintéticos. El control negativo con `IO` debe documentarse por separado para distinguirlo de los casos evaluados bajo la hipótesis de conmutatividad.

A.5. Detalles de Implementación

Esta sección de anexo debe contener los detalles de GHC que serían demasiado densos para el capítulo Implementación de la Solución. Existe para preservar trazabilidad técnica sin interrumpir la lectura principal. Lo ideal es referenciar `docs/renamer.md` y los parches relevantes.

Para completarla, se pueden documentar las funciones principales:

- `rearrangeForApplicativeDo`;
- `isCommutativeQualifiedDo`;
- `buildStmtsDependencyGraph`;
- `generateAllSemanticTopSorts`.

También conviene mencionar los módulos agregados en `base` y `ghc-internal`.